

Chicago Booth Paper No. 26-12

Deep Flow Networks: Optimization-Friendly Surrogates for Integer Predict-then-Optimize

Ozan Candogan

Booth School of Business, University of Chicago

Ayoub Foussoul

Booth School of Business, University of Chicago

Center for Applied Artificial Intelligence
The University of Chicago, Booth School of Business

This paper also can be downloaded without charge from the
Social Science Research Network Electronic Paper Collection

Deep Flow Networks: Optimization-Friendly Surrogates for Integer Predict-then-Optimize

Ozan Candogan

Booth School of Business, University of Chicago

`ozan.candogan@chicagobooth.edu`

Ayoub Foussoul

Booth School of Business, University of Chicago

`ayoub.foussoul@chicagobooth.edu`

Abstract

In the predict-then-optimize framework, historical data are used to train a model that predicts the outcome of a decision in a given context. Upon observing a new context, the trained model is optimized over the feasible decisions. When decisions are integer-valued, standard predictive models can lead to downstream optimization problems that are difficult to solve. We introduce Deep Flow Networks (DFNs), a new class of predictive models designed to combine strong predictive performance with efficient downstream integer optimization. DFNs are inspired by and generalize the minimum-cost flow value functions that map node imbalances to the optimal flow cost. We establish a universality result for DFNs and prove that trained DFNs satisfy a generalization of the classical M -convex exchange property (Murota, 2003), allowing us to characterize the complexity of optimizing them. Guided by these results, we develop a practical implementation of DFNs for learning from data. We test DFNs on a diverse collection of public datasets spanning revenue management, transportation, resource allocation, public services, and manufacturing, and find that their predictive performance is competitive with multi-layer perceptrons, while their downstream optimization problems are solved substantially faster. We also conduct an end-to-end contextual assortment optimization experiment and find that DFNs are fast to optimize and produce the highest-revenue assortments among the learned models.

1 Introduction

We consider the following *predict-then-optimize* problem. A decision maker has access to historical data $\mathcal{D} = \{(x_k, z_k, y_k)\}_{k=1}^K$, where z_k is a context, x_k is a decision, and y_k is the corresponding scalar outcome. The data are used to learn a surrogate $\hat{f}(x, z)$ that predicts the outcome of decision x in context z . After observing a new context z , the decision maker chooses

$$\hat{x}(z) \in \arg \min_{x \in \mathcal{X}(z)} \hat{f}(x, z), \quad (1)$$

where $\mathcal{X}(z)$ is a feasible set.

The predict-then-optimize pipeline commonly arises in two situations. First, the underlying outcome function may be known but difficult to optimize directly. For example, Kaleem et al. (2025) replace the vehicle-routing cost term in a location–routing model with a learned neural-network surrogate and solve the resulting formulation as a MILP. Similarly, Kody et al. (2022) replace the nonlinear AC power-flow constraints in the unit-commitment problem with a learned neural-network surrogate and solve the resulting formulation as a mixed-integer linear program (MILP). In inventory control, Harsha et al. (2025) learn a neural approximation of the cost-to-go and compute replenishment decisions by solving a MILP. However, these approaches often yield MILP reformulations that are large and only weakly structured. As a result, they can still be difficult to solve in practice.

Second, the underlying outcome function may be unknown and must be learned from observed data. In revenue management, for example, one observes customer purchase decisions but not the demand or choice function that generated them. A standard approach is to estimate a parametric demand or choice model, such as a Logit model, and then use the estimated model for pricing or assortment optimization (Talluri and Van Ryzin, 2004; Train, 2009; Besbes and Zeevi, 2009). These models often lead to tractable optimization problems. However, their structural assumptions can limit their ability to represent complex behavior.

These considerations motivate the need for surrogate models that balance approximation power with efficient downstream optimization. For continuous decisions $x \in \mathbb{R}^d$, two broad strategies are common. The first is to learn a surrogate whose structure makes downstream optimization efficient. A representative approach is to learn a surrogate for which $x \mapsto \hat{f}(x, z)$ is convex for every fixed context z . For instance, Calafiore et al. (2020) develop log-sum-exp networks that can approximate any continuous convex function on a compact convex set. Amos et al. (2017) introduce neural-network architectures whose outputs are convex in designated inputs, while Magnani and Boyd (2009) fit convex piecewise-linear models, represented as pointwise maxima of affine functions, by least squares. When $\mathcal{X}(z)$ is a tractable convex set, (1) can then be solved efficiently to global optimality using convex optimization. The second strategy allows a general, possibly nonconvex surrogate and optimizes it using either local or global methods. For example, Trabucco et al. (2021) use gradient-based search to optimize neural-network surrogates. This approach does not provide a global optimality certificate. To obtain global guarantees, Grimstad and Andersson (2019) formulate ReLU networks as mixed-integer linear programs, Mišić (2020) develops mixed-integer formulations for tree ensembles, and Schweidtmann and Mitsos (2019) use deterministic spatial branch-and-bound for neural networks with smooth activation functions. These global methods can certify optimality when they close the optimality gap, but their computational cost can grow quickly with the size of the surrogate.

However, in many applications (including the power generation, vehicle routing, and inventory problems mentioned above) the decisions are inherently discrete (e.g., generator on/off decisions, vehicle routing decisions, or integer replenishment quantities). Both strategies face difficulties in this

setting. For convex surrogates, convexity of $x \mapsto \hat{f}(x, z)$ over $\mathbb{R}^d$ does not ensure that (1) is tractable when $\mathcal{X}(z) \subseteq \mathbb{Z}^d$; even minimizing a convex quadratic objective over $\{0, 1\}^d$ is NP-hard (Gritzmann and Klee, 1989). For general, possibly nonconvex surrogates, gradient-based methods do not directly apply. Exact global formulations can accommodate integer decisions, but these reformulations can lead to large mixed-integer problems that are expensive to solve. Thus, the surrogate classes and optimization methods used for continuous decisions do not by themselves provide efficient downstream optimization over integer domains. To the best of our knowledge, predict-then-optimize methods for integer decisions have typically relied on these same surrogate families, rather than developing surrogates whose structure is specifically designed to facilitate efficient downstream integer optimization while retaining strong approximation power.

In this paper, we propose a new class of function approximators $\hat{f}$ whose structure facilitates efficient downstream optimization over integer decisions while retaining strong approximation power.

1.1 Additional Related Literature

One related strand of literature is on function approximators for set-valued or binary inputs, such as Deep Sets (Zaheer et al., 2017), Deep Submodular Functions (Dolhansky and Bilmes, 2016), and Extended Deep Submodular Functions (Hosseini et al., 2024). While these are discrete-function approximators, they target set-valued or binary domains rather than general integer domains.

Another related strand of literature is on surrogate-optimization models, especially Gaussian-process and Kriging models (Jones et al., 1998), as well as radial-basis-function models (Gutmann, 2001). These models are, however, more commonly used in sequential loops that alternate model fitting, optimization, and new data acquisition than in the fixed-surrogate predict-then-optimize setting considered here.

Our paper is also related to the literature on discrete convex optimization. A convex-extendible discrete function is one that agrees with a convex function at all integer points. Convex extendibility alone is generally too weak to yield interesting optimization guarantees, since minimizing a convex-extendible discrete function over integer points is NP-hard in general, even for convex quadratic objectives over $\{0, 1\}^d$ (Gritzmann and Klee, 1989). Accordingly, several more structured subclasses have been studied, including submodular set functions (Lovász, 1983), $L/L^\natural$ -convex and $M/M^\natural$ -convex functions (Murota, 2003), and integrally convex functions (Favati and Tardella, 1990). Minimum-cost-flow value functions form a particularly important subfamily of M -convex functions (Murota, 2003), combining algorithmic tractability with sufficient richness to approximate complex data. The function approximators proposed in this paper build on these value functions.

A further related literature studies decision-focused learning, in which prediction models are trained using losses derived from the downstream optimization problem. Donti et al. (2017) consider continuous stochastic optimization problems and propagate decision-cost gradients through their solutions to train probabilistic models. Wilder et al. (2019) consider combinatorial problems and make decision-focused training differentiable using continuous relaxations. Elmachet and Grigas (2022) consider problems with predicted linear-objective coefficients and derive the convex SPO+

surrogate for the decision loss. Other approaches differentiate through mixed-integer programs or black-box combinatorial solvers (Ferber et al., 2020; Vlastelica et al., 2020); Mandi et al. (2024) provide a recent review. This literature focuses on incorporating the downstream problem into model training. Our focus is complementary: we design the structure of the predictive model to facilitate downstream integer optimization after training.

A preliminary version of this work appeared in conference proceedings (Candogan and Foussoul, 2026). The present manuscript substantially extends the conference paper by developing a contextual formulation and practical contextual implementation of DFNs, providing a substantially expanded empirical evaluation on public datasets, evaluating the complete predict-then-optimize pipeline in a synthetic contextual assortment-optimization experiment, and adding model diagnostics.

1.2 Contributions

We introduce *Deep Flow Networks* (DFNs), a class of function approximators that is inspired by and generalizes minimum-cost flow value functions that map node imbalances to the optimal flow cost. Given a directed network $G = (V, E)$, edge costs c , edge capacities u , and a vector $\rho \in \mathbb{Z}^p$ specifying the flow balances of a subset $S = \{v_1, \dots, v_p\} \subseteq V$ of nodes, with all other nodes having zero flow balance, the corresponding minimum-cost flow value function is

$$\begin{aligned} \text{val}_G(\rho; c, u) = \min_{\xi \in \mathbb{R}^E} \quad & c^\top \xi \\ \text{s.t.} \quad & \text{div}_{v_i}(\xi) = \rho_i, \quad i \in [p], \\ & \text{div}_v(\xi) = 0, \quad v \in V \setminus S, \\ & 0 \leq \xi_e \leq u_e, \quad e \in E, \end{aligned} \tag{2}$$

where $\text{div}_v(\xi) := \sum_{e=(v,w) \in E} \xi_e - \sum_{e=(w,v) \in E} \xi_e$ denotes the net outflow at node v . DFN functions are functions of the form

$$f(x, z) = \text{val}_G(A(z)x + b(z); c(z), u(z)), \tag{3}$$

where G is a given graph and $c(\cdot)$, $u(\cdot)$, $A(\cdot)$, and $b(\cdot)$ are mappings that may be specified or learned from data, with $c(z)$ and $u(z)$ denoting cost and capacity vectors and $A(z)$ and $b(z)$ denoting an integer matrix and vector, respectively. We show the following properties for DFNs.

Universality. We show that, at any given context z , DFNs can approximate (up to an affine scaling) any function that is convex-extendible in the integer decision x , over any finite set of decisions. The class of convex-extendible functions includes a broad family of important functions both in theory and practice, including set submodular, M/L -convex, and integrally convex functions.

Computational Complexity of Optimizing Trained DFNs. For any fixed context z , if $A(z) = I$, then $x \mapsto f(x, z)$ is an M -convex function that can be minimized efficiently (Murota,

2003). We characterize, for every fixed context z , the computational complexity of minimizing $x \mapsto f(x, z)$ in terms of a parameter $\Delta(A(z))$ that measures the deviation of the learned matrix $A(z)$ from the “easy” M -convex regime ($A(z) = I$). We prove that DFNs satisfy a *generalized exchange property* (Theorem 2), extending the classical M -convex exchange property (Murota, 2003), and use it to bound the optimization complexity as a function of $\Delta(A(z))$ (Lemma 2). Hence, by controlling $\Delta(A(z))$, one can control the complexity of minimizing the trained DFN function for any fixed context z .

Practical Implementation. While our theoretical results apply to DFNs in their most general form, we propose one particular implementation that we found through experimentation to work well in practice. In this implementation, G is a complete bipartite graph with two equally sized node sets, and $u(z) \equiv +\infty$, so all edges are uncapacitated. The two node sets have balance vectors $Wx + h$ and $-(Wx + h)$, respectively, where W is a learned nonnegative integer matrix and h is a learned nonnegative integer vector shared across contexts. Thus, the complete balance matrix and vector are $A = [W^\top, -W^\top]^\top$ and $b = [h^\top, -h^\top]^\top$. We parameterize the edge costs $c(z)$ with a feed-forward neural network. We train the model end to end by differentiating through the minimum-cost flow layer and use straight-through estimation (STE) (Bengio et al., 2013) to learn the integer-valued parameters W and h . We initialize W using a “near-identity” construction. Together, these choices yield a practical implementation of DFNs, while the general framework leaves room to explore alternative implementations for different applications.

Predictive Performance. We assemble a large collection of publicly available datasets from a wide range of operational domains, including revenue management, transportation, resource allocation, public services, and manufacturing, and construct 61 prediction tasks from them. We filter these tasks by training ReLU multi-layer perceptrons on (x, z) and on z alone, retaining the 31 tasks for which x improves prediction beyond the context alone. We then compare DFNs with ReLU multi-layer perceptrons on held-out observations. DFNs provide competitive predictive performance, attaining lower mean test MSE on 13 tasks and remaining within 10% of the MLP on 22 of the 31 tasks. As discussed next, this predictive performance is accompanied by substantially faster downstream optimization.

Downstream Optimization Performance. On the 31 retained tasks, we optimize the validation-selected DFN and MLP over application-specific integer feasible sets. Across 155 optimization problems, the mean solve time is 0.597 seconds for DFN and 761.0 seconds for MLP. The DFN formulation is faster on every task while retaining competitive predictive performance, as reported above.

End-to-End Predict-then-Optimize Performance in Contextual Assortment Optimization. We evaluate the end-to-end predict-then-optimize performance of DFNs on a contextual assortment-optimization problem. In particular, we generate synthetic transaction data from a

simulator and use them to train DFN and several benchmark models that either predict revenue directly or model customer choice. For new customer contexts, we optimize each trained model to select an assortment and use the simulator to evaluate its true expected revenue. After one second, DFN reaches 92.0% of the oracle revenue, compared with 73.1% for the best-performing learned benchmark. After 60 seconds, DFN reaches 92.8%, compared with 81.0% for the best-performing learned benchmark.

Additional Model Diagnostics. We conduct three additional experiments on one of the retained prediction tasks to better understand the practical behavior of DFNs.

- (i) The initialization of A is intended to keep $\Delta(A)$ small and facilitate downstream optimization, while learning A may improve predictive accuracy. We examine this tradeoff by comparing a learned A with one fixed at its initialization. Learning A improves predictive accuracy, whereas fixing A makes downstream optimization even faster while retaining good predictive performance.
- (ii) Our theory links the complexity of optimizing a trained DFN to $\Delta(A)$. We track A and downstream solve time throughout training and find that solve time increases with $\Delta(A)$, consistent with the theory.
- (iii) Learning the integer parameters W and h requires choosing their initialization. We test the robustness of DFNs to this choice and find that their predictive performance remains stable across different initializations.

1.3 Notation

For a positive integer r , let $[r] := \{1, \dots, r\}$. Given a function $f : \mathbb{Z}^d \rightarrow \mathbb{R} \cup \{\pm\infty\}$, its effective domain is $\text{dom } f := \{x \in \mathbb{Z}^d : -\infty < f(x) < +\infty\}$.

For a matrix $A \in \mathbb{Z}^{p \times d}$, let $\text{rank}(A)$ denote the rank of A . If A is square ($p = d$), let $\det(A)$ denote its determinant. In general, let $\det^*(A)$ denote the pseudo-determinant of A , defined as the product of the nonzero singular values of A , with the empty product equal to 1. For $r \leq \min\{p, d\}$, let $\text{diag}_{p,d}(s_1, \dots, s_r)$ denote the $p \times d$ matrix with (i, i) -entry equal to s_i for $1 \leq i \leq r$ and all other entries equal to 0.

For a vector $x \in \mathbb{R}^d$, define the coordinate-wise positive and negative parts by $x^+ := \max\{0, x\}$ and $x^- := \max\{0, -x\}$, where the maxima are taken componentwise. Vector inequalities are also interpreted componentwise. Let e_i denote the i -th standard unit vector, with its dimension determined by context. We write $\|x\|_1 := \sum_i |x_i|$, $\|x\|_\infty := \max_i |x_i|$, and $\|x\|_2 := (\sum_i x_i^2)^{1/2}$ for the ℓ_1 , ℓ_∞ , and Euclidean norms of x , respectively. For indices $1 \leq i \leq j \leq d$, let $x_{i:j} := (x_i, \dots, x_j)^\top \in \mathbb{R}^{j-i+1}$ denote the subvector of x formed by its coordinates $i, \dots, j$.

2 Deep Flow Networks

This section introduces Deep Flow Networks and develops their main theoretical properties. We first define the model. We then establish its approximation power. Finally, we characterize the complexity of optimizing a trained DFN.

2.1 Architecture Definition

A *Deep Flow Network (DFN)* is a tuple $\Pi = (G, c, u, S, A, b)$, where $G = (V, E)$ is a directed graph with $n := |V|$ nodes and $m := |E|$ directed edges; $c : \mathbb{R}^q \rightarrow \mathbb{Q}^m$ is an edge-cost mapping; $u : \mathbb{R}^q \rightarrow \mathbb{Z}_{\geq 0}^m$ is an edge-capacity mapping; $S = \{v_1, \dots, v_p\} \subseteq V$ is a set of $p \geq 2$ special nodes; and $A : \mathbb{R}^q \rightarrow \mathbb{Z}^{p \times d}$ and $b : \mathbb{R}^q \rightarrow \mathbb{Z}^p$ are integer matrix- and vector-valued mappings, respectively.

Given a DFN $\Pi = (G, c, u, S, A, b)$, $z \in \mathbb{R}^q$, and $x \in \mathbb{Z}^d$, let $f(x, z; \Pi)$ denote the optimal value of the minimum-cost flow problem on G with costs $c(z)$, capacities $u(z)$, and where each special node $v_i \in S$ has a net outflow $a_i(z)^\top x + b_i(z)$ (where $a_i(z)^\top$ is the i -th row of $A(z)$ and $b_i(z)$ is the i -th entry of $b(z)$), and every other node $v \in V \setminus S$ has zero net outflow. More precisely, if we let $\text{div}_v(\xi) := \sum_{e \in \delta_v^+} \xi_e - \sum_{e \in \delta_v^-} \xi_e$, where $\delta_v^+ := \{(v, w) \in E\}$ and $\delta_v^- := \{(w, v) \in E\}$, then $f(x, z; \Pi)$ is the optimal value of the integer program

$$\begin{aligned} \min_{\xi} \quad & c(z)^\top \xi \\ \text{s.t.} \quad & \text{div}_{v_i}(\xi) = a_i(z)^\top x + b_i(z), \quad i \in [p], \end{aligned} \tag{4a}$$

$$\text{div}_v(\xi) = 0, \quad v \in V \setminus S, \tag{4b}$$

$$0 \leq \xi_e \leq u_e(z), \quad \xi_e \in \mathbb{Z}, \quad e \in E. \tag{4c}$$

We adopt the convention that $f(x, z; \Pi) = +\infty$ if (4) is infeasible. The constraint matrix of the linear relaxation of (4) is totally unimodular. Since its right-hand side and capacities are integral, every extreme point is integral, so solving the linear relaxation solves (4). It follows that (4) can be solved in polynomial time using linear-programming algorithms; its network structure also permits specialized minimum-cost flow algorithms (Ahuja et al., 1993). As we will see in our implementation of DFNs for learning from data (Section 3), $f(x, z; \Pi)$ serves (up to an affine scaling) as the learned surrogate $\hat{f}(x, z)$ in the predict-then-optimize problem (1), where z is the context, x is the integer decision, and the mappings c , u , A , and b may be specified or learned from data.

2.2 Universality

In this section, we study the approximation power of DFN functions. The following theorem shows that they can approximate (up to an affine scaling) a broad class of discrete ground-truth functions.

Theorem 1. *Consider a function $g : \mathbb{R}^d \times \mathbb{R}^q \rightarrow \mathbb{R}$ such that $x \mapsto g(x, z)$ is convex for every $z \in \mathbb{R}^q$. Given a finite set of integers $I \subset \mathbb{Z}^d$ and $\epsilon > 0$, there exist a DFN Π and functions $\alpha, \beta : \mathbb{R}^q \rightarrow \mathbb{R}$*

such that

$$|g(x, z) - (\alpha(z)f(x, z; \Pi) + \beta(z))| \leq \epsilon$$

for every $z \in \mathbb{R}^q$ and every $x \in I$.

The proof of Theorem 1 is given in Appendix A.1. A discrete function is *convex-extendible* if it agrees with a convex function at every integer point. Theorem 1 shows that, on a finite integer domain and up to an affine scaling, DFN functions can approximate any ground-truth function that is convex-extendible in the decision for every context. The class of convex-extendible functions includes many important functions that arise in practice, including submodular set functions (Lovász, 1983), M - and $M^\natural$ -convex functions (Murota, 2003), L - and $L^\natural$ -convex functions (Murota, 2003), and integrally convex functions (Favati and Tardella, 1990).

2.3 Computational Complexity of Optimizing Trained DFNs

Fix a DFN $\Pi = (G, c, u, S, A, b)$ and an arbitrary context $z \in \mathbb{R}^q$. Let $c_z := c(z)$, $u_z := u(z)$, $A_z := A(z)$, $b_z := b(z)$, and $f_z(x) := f(x, z; \Pi)$. The goal of this section is to characterize the complexity of minimizing f_z over its effective domain as a function of its parameters. In particular, we define a parameter $\Delta(A_z)$ which depends on the matrix A_z and show that the complexity of minimizing f_z scales with $\Delta(A_z)$.

When $A_z = I$, the function f_z is known to be M -convex. In particular, it satisfies the following exchange property:

Lemma 1 (EXC). *For every $x, x' \in \text{dom } f_z$ and every $i \in [p]$ with $x_i > x'_i$, there exists $j \in [p]$ with $x_j < x'_j$ such that*

$$f_z(x) + f_z(x') \geq f_z(x - e_i + e_j) + f_z(x' + e_i - e_j)$$

In words, whenever x has a unit surplus over x' in some coordinate i , there exists a coordinate j in which x has a unit deficit relative to x' such that performing the elementary swap $\tilde{x} = x - e_i + e_j$ on x (and the opposite swap on x'), thereby bringing the two points closer, keeps both points feasible and does not increase the sum of their objective values. This can be viewed as a discrete analogue of classical convexity where moving two points toward their “average” does not increase the sum of their values. It is also analogous to the classical exchange axiom of matroids; see Oxley (2011). A proof of Lemma 1 can be found in Murota (2003); see also Lemma 3 where we prove a stronger exchange property for this class of functions. As a consequence, f_z can be minimized efficiently. In particular, Shioura (2022) shows that the steepest descent algorithm starting at any feasible solution and iteratively selecting the best-improving neighbor among points $x + e_i - e_j$, $i, j \in [p]$ reaches a global minimum of f_z in time polynomial in the input size and in the ℓ_1 -distance from the initial point to a nearest minimizer.

When A_z is a general integer matrix, however, f_z need not be M -convex, and the steepest-descent algorithm of Shioura (2022) is no longer guaranteed to find a global minimizer of f_z . We show below that f_z satisfies a generalized exchange property with exchange size bounded by $\Delta(A_z)$,

where $\Delta(A_z) = 2$ when $A_z = I$ (recovering the exchange property in Lemma 1) and, more generally, measures departure from this M -convex case. Based on this property, we establish a local descent algorithm similar to that of Shioura (2022) in which each iteration moves to an improving point in a neighborhood whose size depends on $\Delta(A_z)$, then link its time complexity to the magnitude of the parameter $\Delta(A_z)$.

Let $k = p - \text{rank}(A_z)$ denote the (row) corank of A_z and let

$$\Delta(A_z) = 2 \cdot (2k)^{2k} \cdot \det^*(A_z)$$

where $\Delta(A_z) = 2 \det^*(A_z)$ when $k = 0$, i.e., when A_z is full row rank. The function f_z has the following generalized exchange property (G-EXC).

Theorem 2 (G-EXC). *For every $x, x' \in \text{dom } f_z$ such that $A_z x \neq A_z x'$, there exists $\tilde{x} \in \mathbb{Z}^d$ such that,*

$$f_z(x) + f_z(x') \geq f_z(x - \tilde{x}) + f_z(x' + \tilde{x}).$$

Moreover, $0 < \|A_z \tilde{x}\|_1 \leq \Delta(A_z)$, $(A_z \tilde{x})^+ \leq (A_z x - A_z x')^+$, and $(A_z \tilde{x})^- \leq (A_z x - A_z x')^-$.

The proof of Theorem 2 is given in the next subsection (Section 2.4). Given the generalized exchange property (G-EXC), the global minimum of f_z can be found using the local descent Algorithm 1.

Algorithm 1 Local Descent

```

Initialize:  $x \in \text{dom } f_z$ .
while true do
   $\tilde{x} \leftarrow 0$ 
  for  $y \in \mathbb{Z}^p$  s.t.  $\|y\|_1 \leq \Delta(A_z)$  do
    if  $\exists \hat{x} \in \mathbb{Z}^d$  s.t.  $A_z \hat{x} = y$  and  $f_z(x + \hat{x}) < f_z(x)$  then
      Choose any such  $\hat{x}$ ; set  $\tilde{x} \leftarrow \hat{x}$ ; break
    end if
  end for
  if  $\tilde{x} = 0$  then
    return  $x$ 
  end if
   $x \leftarrow x + \tilde{x}$ 
end while

```

In particular, the following lemma holds:

Lemma 2. *Algorithm 1 finds a global minimizer of f_z and terminates in time polynomial in $p^{\lceil \Delta(A_z) \rceil}$, n , m , d , D_z , $\|c_z\|_\infty$, $\|u_z\|_\infty$, $\log(1 + \|A_z\|_\infty)$, and $\log(1 + \|b_z\|_\infty)$, where D_z is the smallest positive integer such that $D_z \cdot (c_z)_e \in \mathbb{Z}$ for every $e \in E$.*

The proof of Lemma 2 is given in Appendix A.2. Lemma 2 shows that the complexity of minimizing f_z is closely related to the parameter $\Delta(A_z)$. This relationship can be leveraged in

supervised regression with DFN functions to learn approximators that are easy to optimize over integer domains.

2.4 Proof of Theorem 2

Let $f_z^* : \mathbb{Z}^p \rightarrow \mathbb{R} \cup \{+\infty\}$ denote the minimum-cost flow value function obtained from f_z by replacing A_z with the identity matrix I_p , while keeping G, c_z, u_z, S , and b_z fixed. In particular, $f_z(x) = f_z^*(A_z x)$ for every $x \in \mathbb{Z}^d$. As mentioned earlier, f_z^* is M -convex and therefore satisfies the exchange property in Lemma 1.

We next show that f_z^* in fact satisfies an even stronger exchange property, which we call the *strong base-orderable exchange (SBO-EXC)* property. This is an analogue of *strong base-orderability* for matroids, where a matroid is said strongly base-orderable if, for any two bases B and B' , there exists a bijection $\psi : B \rightarrow B'$ such that for every subset $X \subseteq B$, both $(B \setminus X) \cup \psi(X)$ and $(B' \setminus \psi(X)) \cup X$ are bases (Oxley, 2011).

Lemma 3 (SBO-EXC). *For any distinct $y, y' \in \text{dom } f_z^*$, there exist indices $i_1, \dots, i_K$ and $j_1, \dots, j_K$ in $[p]$ such that*

$$\begin{aligned} (y - y')^+ &= \sum_{t=1}^K e_{i_t}, \\ (y - y')^- &= \sum_{t=1}^K e_{j_t}. \end{aligned}$$

Letting $\tilde{y}_t := e_{i_t} - e_{j_t}$ for $t = 1, \dots, K$, and $\tilde{y}_T := \sum_{t \in T} \tilde{y}_t$ for every $T \subseteq [K]$, it also holds that,

$$\begin{aligned} f_z^*(y) + f_z^*(y') &\geq f_z^*(y - \tilde{y}_T) \\ &\quad + f_z^*(y' + \tilde{y}_T). \end{aligned}$$

In words, Lemma 3 says that the unit surpluses of y over y' can be paired with the unit deficits of y relative to y' to obtain K elementary swaps $\tilde{y}_t = e_{i_t} - e_{j_t}$, such that applying any subset of the swaps to y (and the opposite swaps to y') does not increase the sum of the objective values of the two points. The proof of Lemma 3 is given in Appendix A.3.

Now, let $x, x' \in \text{dom } f_z$ such that $A_z x \neq A_z x'$, and set $y := A_z x$ and $y' := A_z x'$. Let $\tilde{y}_1, \dots, \tilde{y}_K$ denote the elementary swaps given by Lemma 3 applied to $y, y' \in \text{dom } f_z^*$. Assume there exists $|T| \leq \Delta(A_z)/2$ and $\tilde{x} \in \mathbb{Z}^d$ such that $A_z \tilde{x} = \tilde{y}_T$, then

$$A_z(x - \tilde{x}) = y - \tilde{y}_T, \quad A_z(x' + \tilde{x}) = y' + \tilde{y}_T,$$

and by Lemma 3,

$$f_z(x) + f_z(x') \geq f_z(x - \tilde{x}) + f_z(x' + \tilde{x}).$$

Moreover, since each swap satisfies $\|\tilde{y}_t\|_1 = 2$, we have $\|A_z \tilde{x}\|_1 = \|\tilde{y}_T\|_1 \leq 2|T| \leq \Delta(A_z)$. Also, $(A_z \tilde{x})^+ = (\tilde{y}_T)^+ \leq (y - y')^+$ and $(A_z \tilde{x})^- = (\tilde{y}_T)^- \leq (y - y')^-$, implying the theorem. The goal

henceforth is therefore to find a subset T of cardinality $|T| \leq \Delta(A_z)/2$ such that the system of Diophantine equations $A_z \tilde{x} = \tilde{y}_T$ has a solution over integers $\tilde{x} \in \mathbb{Z}^d$.

The standard method to solve a system of Diophantine equations uses the Smith normal form (SNF) of A_z .

Definition 1 (Smith Normal Form). Let $A_z \in \mathbb{Z}^{p \times d}$ with $r = \text{rank}(A_z)$. Then A_z admits a factorization $A_z = VDU$, where $V \in \mathbb{Z}^{p \times p}$ and $U \in \mathbb{Z}^{d \times d}$ are unimodular integer matrices (i.e. $\det(U) = \pm 1$ and $\det(V) = \pm 1$), and

$$D = \text{diag}_{p,d}(s_1, \dots, s_r) \in \mathbb{Z}^{p \times d},$$

where $s_i \geq 1$ for every $i = 1, \dots, r$, and s_i divides s_{i+1} for every $i = 1, \dots, r-1$.

Given an SNF $A_z = VDU$, the following claim, whose proof is immediate, characterizes when the system has an integer solution.

Claim 1. *The system $A_z \tilde{x} = \tilde{y}_T$ has a solution over integers $\tilde{x} \in \mathbb{Z}^d$ iff*

$$(V^{-1} \tilde{y}_T)_i \text{ is a multiple of } s_i \quad \forall i \in \{1, \dots, r\}.$$

and

$$(V^{-1} \tilde{y}_T)_{r+1:p} = 0.$$

Now, the system has a solution for $T = [K]$, since $\tilde{y}_{[K]} = y - y' = A_z(x - x')$.

Suppose first that $k = 0$, and hence $r = p$. It is known that $\prod_{i=1}^p s_i$ is the greatest common divisor of the $p \times p$ minors of A_z (Schrijver, 1986), while the Cauchy–Binet identity applied to $A_z A_z^\top$ shows that $\det^*(A_z)$ is the square root of the sum of their squares. Hence,

$$\prod_{i=1}^p s_i \leq \det^*(A_z) = \frac{\Delta(A_z)}{2}.$$

Set $\delta_0 := 0$, and define the prefix sums $\delta_t := \sum_{s=1}^t V^{-1} \tilde{y}_s \in \mathbb{Z}^p$ for $t = 1, \dots, K$. We view δ_t in the product group

$$(\mathbb{Z}/s_1\mathbb{Z}) \times \dots \times (\mathbb{Z}/s_p\mathbb{Z}).$$

This group has exactly $\prod_{i=1}^p s_i$ elements. Therefore, either $K \leq \prod_{i=1}^p s_i$, in which case we take $T = [K]$, or $K > \prod_{i=1}^p s_i$, in which case among the first $1 + \prod_{i=1}^p s_i$ prefix sums there exist indices $0 \leq \ell < t \leq \prod_{i=1}^p s_i$ such that $\delta_\ell = \delta_t$ in the product group. In the latter case,

$$\sum_{s=\ell+1}^t V^{-1} \tilde{y}_s = 0.$$

in the product group above. By Claim 1, there exists $\tilde{x} \in \mathbb{Z}^d$ such that $A_z \tilde{x} = \sum_{s=\ell+1}^t \tilde{y}_s$, and we

take $T := \{\ell + 1, \dots, t\}$. In both cases,

$$|T| \leq \prod_{i=1}^p s_i \leq \frac{\Delta(A_z)}{2}.$$

We therefore assume $k \geq 1$ below.

By Claim 1,

$$\sum_{t=1}^K (V^{-1} \tilde{y}_t)_{r+1:p} = (V^{-1} \tilde{y}_{[K]})_{r+1:p} = 0.$$

We now apply the following lemma, also known as Steinitz' lemma, to reorder the vectors

$$(V^{-1} \tilde{y}_1)_{r+1:p}, \dots, (V^{-1} \tilde{y}_K)_{r+1:p},$$

whose sum is zero, in a way that all the resulting prefix sums have a norm bounded by a quantity that is independent of the number of vectors K , implying that for a large enough K , two prefix sums must be equal and hence their difference gives a subset of these vectors whose sum is zero.

Lemma 4 (see, e.g., Bárány (2008)). *Let $(\mathbb{R}^k, \|\cdot\|)$ be any k -dimensional normed space with unit ball $B := \{z \in \mathbb{R}^k : \|z\| \leq 1\}$. If $v_1, \dots, v_K \in B$ and $\sum_{i=1}^K v_i = 0$, then there exists a permutation π of $\{1, \dots, K\}$ such that for every $t = 1, \dots, K$,*

$$\left\| \sum_{i=1}^t v_{\pi(i)} \right\| \leq k.$$

For every $i = r + 1, \dots, p$, define

$$M_i := \frac{1}{2} \max_{a,b=1,\dots,p} |(V^{-1})_{ia} - (V^{-1})_{ib}| + \frac{1}{4k}.$$

Let

$$R := \text{diag}\left(\frac{1}{2M_{r+1}}, \dots, \frac{1}{2M_p}\right).$$

Note that for every $t = 1, \dots, K$ and $i = r + 1, \dots, p$,

$$|(V^{-1} \tilde{y}_t)_i| = |(V^{-1} e_{it} - V^{-1} e_{jt})_i| \leq 2M_i.$$

Hence, for every $t = 1, \dots, K$,

$$\|R(V^{-1} \tilde{y}_t)_{r+1:p}\|_\infty \leq 1.$$

Applying Steinitz' lemma to the vectors $v_t := R(V^{-1} \tilde{y}_t)_{r+1:p} \in \mathbb{R}^k$ in the normed space $(\mathbb{R}^k, \|\cdot\|_\infty)$, there exists a permutation π of $[K]$ such that for every $t = 1, \dots, K$,

$$\left\| \sum_{s=1}^t v_{\pi(s)} \right\|_\infty \leq k.$$

Equivalently, for every $t = 1, \dots, K$ and $i = r+1, \dots, p$,

$$-2kM_i \leq \sum_{s=1}^t (V^{-1}\tilde{y}_{\pi(s)})_i \leq 2kM_i.$$

Set $\delta_0 := 0$, and define the prefix sums $\delta_t := \sum_{s=1}^t V^{-1}\tilde{y}_{\pi(s)} \in \mathbb{Z}^p$ for $t = 1, \dots, K$. We view δ_t in the product group

$$\left(\mathbb{Z}/s_1\mathbb{Z}\right) \times \dots \times \left(\mathbb{Z}/s_r\mathbb{Z}\right) \times \mathbb{Z}^k.$$

i.e., the first r coordinates of δ_t are taken modulo $(s_1, \dots, s_r)$ and the last k coordinates are kept in $\mathbb{Z}^k$. Since $\mathbb{Z}/s_1\mathbb{Z} \times \dots \times \mathbb{Z}/s_r\mathbb{Z}$ has exactly $\prod_{i=1}^r s_i$ elements, and since $2kM_i$ is a half-integer, coordinate i of δ_t can take at most $4kM_i$ integer values for every $i = r+1, \dots, p$, the number of possible values of δ_t in the product group is at most

$$N := \left(\prod_{i=1}^r s_i\right) \cdot (4k)^k \cdot \prod_{i=r+1}^p M_i.$$

Therefore, either $K \leq N$, in which case we take $T = [K]$, or $K > N$, in which case among the $N+1$ prefix sums $\delta_0, \dots, \delta_N$, there exist indices $0 \leq \ell < t \leq N$ such that $\delta_\ell = \delta_t$ in the product group. In the latter case,

$$\sum_{s=\ell+1}^t V^{-1}\tilde{y}_{\pi(s)} = 0$$

$$\text{in } \left(\mathbb{Z}/s_1\mathbb{Z}\right) \times \dots \times \left(\mathbb{Z}/s_r\mathbb{Z}\right) \times \mathbb{Z}^k.$$

By Claim 1, this implies that $\sum_{s=\ell+1}^t \tilde{y}_{\pi(s)}$ lies in the integer image of A_z , i.e., there exists $\tilde{x} \in \mathbb{Z}^d$ such that

$$A_z \tilde{x} = \sum_{s=\ell+1}^t \tilde{y}_{\pi(s)},$$

and we pick $T := \{\pi(\ell+1), \dots, \pi(t)\}$. In both cases, $|T| \leq N$ and we have a solvable system $A_z \tilde{x} = \tilde{y}_T$.

We conclude by showing the following lemma.

Lemma 5. *Suppose $k \geq 1$. The Smith normal form $A_z = VDU$ can be chosen such that*

$$\prod_{i=r+1}^p M_i \leq k^k \cdot \frac{\det^*(A_z)}{\prod_{i=1}^r s_i}.$$

The proof of the lemma is given in Appendix A.4.

3 Implementation

We now present a particular implementation of DFNs for learning from data that we found through experimentation to work well in practice. The general theoretical framework developed in Section 2, however, leaves room to explore alternative implementations for different applications.

3.1 Model Architecture

We consider surrogates of the form

$$\widehat{f}(x, z) = \alpha(z)f(x, z; \Pi) + \beta(z),$$

where $\Pi = (G, c, u, S, A, b)$ is a DFN and α and β are learned scalar-valued mappings of the context. We next specify the components of Π and the mappings α and β used in our implementation.

Graph Topology. We take G to be a complete directed bipartite graph with equally many left and right nodes. There is an arc from every left node to every right node. The number of nodes on each side is selected by validation.

Costs and Capacities. We take the cost mapping $z \mapsto c(z)$ to be a multilayer perceptron with ReLU activations and set $u(z) = +\infty$ for every context z (or, equivalently, to a sufficiently large finite constant). The depth and width of the cost network are selected by validation.

Balances. For an input x , the balance vector of the left nodes is $Wx + h$, and that of the right nodes is $-(Wx + h)$, where W is a learned nonnegative integer matrix and h is a learned nonnegative integer vector, both shared across contexts. The complete balance matrix and vector are therefore $A = [W^\top, -W^\top]^\top$ and $b = [h^\top, -h^\top]^\top$, so that the full balance vector is $Ax + b$. Its entries sum to zero for every x . Since W and h are nonnegative, $Wx + h$ is nonnegative for every nonnegative input x , and the complete uncapacitated graph then admits a feasible flow. Any input domain with finite coordinatewise lower bounds can be translated so that its inputs are nonnegative before they enter the model. The minimum-cost flow problem defining $f(x, z; \Pi)$ is therefore feasible for every nonnegative input x and every context z and, in particular, reduces to a balanced transportation problem.

Affine Mappings. We use two separate single-hidden-layer perceptrons with ReLU activations to define the mappings $z \mapsto \alpha(z)$ and $z \mapsto \beta(z)$. Their hidden-layer widths are selected by validation. Note that the affine scaling does not change the complexity of optimizing $\widehat{f}(\cdot, z)$ for any fixed context z . In particular, we constrain $\alpha(z) > 0$ when the downstream problem is a minimization problem and $\alpha(z) < 0$ when it is a maximization problem by applying, respectively, a positive or negative softplus transformation to the output of the network defining α .

3.2 Training

Given a dataset $\mathcal{D} = \{(x_k, z_k, y_k)\}_{k=1}^K$, we train $\hat{f}$ by minimizing a training objective $\mathcal{L}(\hat{f}; \mathcal{D})$ over W , h , and the parameters of the networks defining c , α , and β . We next describe how gradients are obtained through the minimum-cost flow problem, how the integer parameters W and h are learned, and how they are initialized.

Differentiation Through Min-Cost Flow. Because $\hat{f}(x, z)$ depends on the model parameters through the optimal value $f(x, z; \Pi)$, minimizing the training objective with gradient-based methods requires differentiating through the minimum-cost flow problem defining $f(x, z; \Pi)$. For each training example (x, z) , we solve the associated minimum-cost flow problem (4) using a specialized transportation solver and obtain an optimal primal solution ξ^* along with optimal dual variables for the net outflow constraints. The optimal flow and dual variables provide subgradients with respect to the arc costs and node balances, which are used to train the model.

Learning Integer Parameters via Straight-Through Estimation (STE). Because W and h are constrained to be integer-valued, we use straight-through estimation (STE) (Bengio et al., 2013). Moreover, since W and h must be nonnegative, we apply a softplus map to enforce nonnegativity. Concretely, we maintain real-valued parameters $(\bar{W}, \bar{h})$. In the forward pass, we form the integer parameters used by the min-cost flow solver as $W = \text{round}(\text{softplus}(\bar{W}))$ and $h = \text{round}(\text{softplus}(\bar{h}))$. In the backward pass, STE treats the rounding operator $\text{round}(\cdot)$ as the identity map, allowing gradients to propagate to $(\bar{W}, \bar{h})$ through the softplus map.

Initialization and Keeping $\Delta(A)$ Small. To encourage a small value of $\Delta(A)$, we choose W according to a “near-identity” construction that cycles through the standard basis vectors in round-robin order: the rows of W are set to $e_1, e_2, \dots, e_d, e_1, e_2, \dots$ until all rows are filled. We initialize h to $\mathbf{0}$.¹

4 Predictive Performance

In this section, we evaluate the predictive performance of DFNs on a diverse collection of real-data tasks with integer-valued inputs. We compare DFNs with multi-layer perceptrons (MLPs) across these tasks.

4.1 Dataset Selection

We assembled a large collection of publicly available datasets in revenue management, transportation, resource allocation, public services, manufacturing, and related operational domains that contain integer-valued inputs, contextual information, and scalar outcomes. From these datasets, we

¹We obtain this initialization by setting the relevant entries of $\bar{W}$ to $\log(e - 1)$ and all remaining entries of $\bar{W}$ and $\bar{h}$ to -3 .

constructed 61 prediction tasks. Each task consists of observations $\mathcal{D} = \{(x_k, z_k, y_k)\}_{k=1}^K$, where $x_k \in \mathbb{Z}^d$ is an integer-valued input, z_k is a context, and y_k is a scalar outcome. Table B.1 in the Appendix provides the complete list of tasks and their sources.

Next, we filter the 61 tasks to retain those for which x improves prediction beyond the information contained in z . Specifically, for each task, we train one fully connected ReLU MLP on (x, z) and another on z alone. For each model, we sample 12 distinct hyperparameter configurations from the grid reported in Table B.2 in the Appendix, train each configuration with Adam for at most 300 epochs by minimizing MSE on the standardized outcome, use an early-stopping patience of 50 epochs, and retain the checkpoint with the lowest validation standardized MSE. We retain a task if the (x, z) model has lower mean validation standardized MSE than the z -only model across five random seeds. This procedure retains 31 tasks and ensures that the integer input contributes predictive information beyond the context.

Table B.3 in the Appendix describes the 31 retained tasks, including their integer inputs, contexts, outcomes, dimensions, sample sizes, and training, validation, and test splits. The collection spans allocation, scheduling, pricing, inventory, composition, and process-design tasks. The datasets contain between 99 and 156,965 observations, with integer-input dimensions between 5 and 96 and context dimensions between 1 and 555.

4.2 Models and Training Protocol

We compare the following two models.

MLP. The benchmark is a fully connected neural network with ReLU activations. It takes the concatenated input (x, z) and returns a scalar prediction. Before training, we preprocess the data using the training split only. For each numerical context variable, we replace missing values with the training median, then subtract the training mean and divide by the training standard deviation. We convert each categorical context variable into binary indicators, one for each category observed in the training split. We standardize each coordinate of the integer input x and the outcome in the same way. For each retained task, we draw 12 distinct MLP configurations uniformly at random from the candidate values reported in Table B.2 in the Appendix and use the same 12 configurations for each of five training seeds. For each seed, we train every configuration with Adam for at most 300 epochs by minimizing MSE on the standardized outcome, using mini-batches of 1,024 and a constant learning rate. Within each training run, we use early-stopping patience 50 and retain the checkpoint with the lowest validation standardized MSE. We then select, separately for each seed, the retained checkpoint with the lowest validation standardized MSE across the 12 configurations. This selection determines the depth, width, learning rate, and weight decay. The selected checkpoint is evaluated once on the test split.

DFN. We use the practical DFN implementation described in Section 3, with learned integer parameters (W, h) , context-dependent transportation costs, and context-dependent affine output

scaling. The sign of the output scale is chosen according to whether the outcome is minimized or maximized. Before training, we preprocess the context and outcome using the training split only. For each numerical context variable, we replace missing values with the training median, then subtract the training mean and divide by the training standard deviation. We convert each categorical context variable into binary indicators, one for each category observed in the training split. We standardize the outcome in the same way and leave the integer input x unchanged. For each retained task, we draw 12 distinct DFN configurations uniformly at random from the candidate values reported in Table B.2 and use the same 12 configurations for each of five training seeds. For each seed, we train every configuration with Adam for at most 300 epochs by minimizing MSE on the standardized outcome. We use mini-batches of 512 when the transportation graph has at most 32 nodes on each side, 256 when it has between 33 and 64, and 128 when it has more than 64, together with a constant learning rate. Within each training run, we use early-stopping patience 50 and retain the checkpoint with the lowest validation standardized MSE. We then select, separately for each seed, the retained checkpoint with the lowest validation standardized MSE across the 12 configurations. This selection determines the number of transportation nodes, the cost-network architecture, the learning rate, the weight decay, the integer-parameter learning-rate multiplier, and the initial output scale. The selected checkpoint is evaluated once on the test split.

4.3 Software and Hardware

We use Python 3.13.5, PyTorch 2.8.0, and NumPy 2.3.4 on a MacBook Pro with an Apple M4 Max chip, 14 CPU cores, and 36 GB of RAM. To solve the transportation problems that arise during DFN training, we use the specialized `ot.emd` transportation solver from POT 0.9.6.post1.

4.4 Results

Table 1 compares DFN and MLP on all 31 retained tasks. DFN achieves lower mean test MSE on 13 tasks and is within 10% of the MLP on 9 additional tasks. Thus, DFN either outperforms the MLP or remains within 10% of it on 22 of the 31 tasks.

These results show that DFNs are competitive with ReLU MLPs across a diverse collection of real-data tasks. As shown in Section 5, the resulting downstream optimization is substantially faster for DFN than for the ReLU MLP. DFNs therefore combine competitive predictive performance with the optimization-friendly structure needed for efficient downstream integer optimization.

5 Downstream Optimization

In this section, we evaluate the downstream optimization performance of trained DFNs. We compare the time required to optimize DFNs and MLPs across the tasks considered in Section 4.

In particular, we consider the 31 datasets retained by the filtering procedure in Section 4.1. For each task, we use the validation-selected DFN and MLP from training seed zero and sample five

Table 1: Standardized Test MSE on the 31 Retained Tasks

Task	MLP	DFN
Bay Wheels	1.195 ± 0.031	0.980 ± 0.064
Bicing	0.746 ± 0.018	0.758 ± 0.017
Bluebikes	0.603 ± 0.007	0.663 ± 0.031
BPI 2019 purchase orders	0.759 ± 0.023	0.824 ± 0.044
BTS airport scheduling	0.446 ± 0.023	0.363 ± 0.024
Capital Bikeshare	0.355 ± 0.013	0.293 ± 0.011
Chemical manufacturing	0.518 ± 0.019	1.216 ± 0.104
CMS nursing homes	0.478 ± 0.003	0.492 ± 0.001
Criteo campaign allocation	0.0297 ± 0.0018	0.0356 ± 0.0024
Dominick’s cheese demand	0.797 ± 0.132	0.403 ± 0.007
Dominick’s cheese revenue	1.118 ± 0.162	0.408 ± 0.019
Dominick’s frozen-dinner revenue	0.306 ± 0.004	0.332 ± 0.004
Favorita promotions, log-lag context	0.216 ± 0.007	0.203 ± 0.003
Favorita promotions, raw-lag context	0.211 ± 0.004	0.230 ± 0.007
Genpact food demand	0.216 ± 0.006	0.254 ± 0.019
Grain-legume management	0.620 ± 0.002	0.624 ± 0.004
iFLEX electricity pricing	0.0354 ± 0.0007	0.0363 ± 0.0028
Retail inventory	0.644 ± 0.008	0.516 ± 0.006
MIND news slate	0.864 ± 0.002	0.861 ± 0.003
Food delivery	0.0908 ± 0.0010	0.0840 ± 0.0004
NYC taxi positioning	0.0362 ± 0.0008	0.0485 ± 0.0018
Olist fulfillment	0.095 ± 0.003	0.324 ± 0.086
Open University learning allocation	0.751 ± 0.001	0.777 ± 0.002
Planned delivery routes	0.561 ± 0.014	0.538 ± 0.014
SF fire dispatch	0.756 ± 0.002	0.742 ± 0.006
SFpark pricing	0.761 ± 0.023	0.926 ± 0.131
Technical-gadget retail	2.586 ± 0.100	2.813 ± 0.210
Bank contact planning	5.183 ± 0.162	8.072 ± 1.115
Concrete mixture design	0.0873 ± 0.0070	0.1495 ± 0.0074
Garment production	24.492 ± 4.437	4.093 ± 1.907
Superconductor design	0.1176 ± 0.0113	0.0996 ± 0.0011

Note. Entries are mean $\pm$ standard error over five random seeds.

contexts uniformly without replacement from the test split. At each sampled context, we formulate the downstream optimization problem as

$$\min / \max_{x \in \mathcal{X}(z)} \widehat{f}(x, z), \quad (5)$$

Here, $\mathcal{X}(z)$ is the set of feasible decisions for context z . The choice between minimization and maximization follows the outcome being predicted in each task.

5.1 Feasible Sets

We use a simple feasible set tailored to each task. In allocation tasks, we keep the total number of units equal to its observed value and decide how many units to assign to each location, product, or category. In pricing tasks, each product has a finite set of allowed prices, and the decision selects one price for each product. The allowed prices are those observed in the training data. Promotion and placement tasks select which products to promote or display, subject to a limit on how many may be selected. In categorical tasks, exactly one option is selected from each group. In routing tasks, the decision specifies the order in which the stops are visited. In composition and design tasks, decision variables remain within their observed ranges and satisfy constraints on quantities such as the total mixture, the number of constituent elements, or the available labor. Table B.4 in the Appendix gives the exact feasible sets.

5.2 Optimization Formulations

DFN. For a fixed context z , optimizing a trained DFN over $x \in \mathcal{X}(z)$ is equivalent to a MILP. We obtain this formulation by substituting the transportation problem defining the DFN into (5). The full formulation appears in Appendix B.1.5.

MLP. For a fixed context z , optimizing a trained ReLU MLP over $x \in \mathcal{X}(z)$ is also equivalent to a MILP. We represent each ReLU using the standard exact big- M formulation (Tjeng et al., 2019). The full formulation appears in Appendix B.1.5.

5.3 Software and Hardware

We use Python 3.13.5, PyTorch 2.8.0, NumPy 2.3.4, and Gurobi 12.0.3 on a MacBook Pro with an Apple M4 Max chip, 14 CPU cores, and 36 GB of RAM.

5.4 Results

Each solve has a 15-minute limit. Table 2 reports the elapsed Gurobi runtime; runs that do not terminate earlier use the full time limit.

Gurobi terminates earlier for the DFN formulation on every task. Averaged over all 155 optimization problems, the runtime is 0.597 seconds for DFN and 761.0 seconds for MLP. The DFN

Table 2: Mean Gurobi Runtime over Five Sampled Test Contexts, in Seconds, with a 15-Minute Limit per Solve

Task	DFN	MLP
Bay Wheels	0.909	900.4
Bicing	0.073	900.0
Bluebikes	0.220	900.0
BPI 2019 purchase orders	0.064	540.3
BTS airport scheduling	0.059	900.0
Capital Bikeshare	1.562	901.7
Chemical manufacturing	0.008	900.1
CMS nursing homes	0.006	629.3
Criteo campaign allocation	0.013	900.7
Dominick’s cheese demand	0.014	900.1
Dominick’s cheese revenue	0.743	900.1
Dominick’s frozen-dinner revenue	0.076	900.0
Favorita promotions, log-lag context	1.434	867.7
Favorita promotions, raw-lag context	1.576	21.1
Genpact food demand	0.131	811.3
Grain-legume management	0.436	766.2
iFLEX electricity pricing	0.020	396.0
Retail inventory	0.315	900.1
MIND news slate	0.335	675.2
Food delivery	0.079	422.4
NYC taxi positioning	2.556	900.2
Olist fulfillment	0.085	545.6
Open University learning allocation	1.373	902.1
Planned delivery routes	0.116	272.1
SF fire dispatch	0.004	900.1
SFpark pricing	0.222	900.0
Technical-gadget retail	0.080	900.0
Bank contact planning	0.034	713.1
Concrete mixture design	0.013	900.0
Garment production	0.001	900.2
Superconductor design	5.940	725.3

mean runtime is below one second on 25 of the 31 tasks and below six seconds on every task. The MLP mean runtime exceeds 400 seconds on 28 tasks and 890 seconds on 18 tasks. Together with the predictive results in Section 4.4, these findings show that DFNs combine competitive predictive performance with substantially faster downstream integer optimization.

6 End-to-End Predict-Then-Optimize Performance in Contextual Assortment Optimization

In this section, we evaluate the end-to-end predict-then-optimize performance of DFNs in a contextual assortment-optimization experiment. We compare the quality of the assortments obtained using DFNs for prediction then optimization and those obtained using several benchmark models.

While Sections 4 and 5 evaluate the predictive performance and downstream optimization time of DFNs, they do not show whether optimizing the learned models leads to high-quality decisions. The datasets used in those experiments record outcomes only for observed decisions, so they do not show how the decisions obtained by optimizing the trained models would perform. In this section, we consider a simulated environment that allows us to evaluate the counterfactual outcomes of the assortments selected by the different models and hence assess their end-to-end predict-then-optimize performance.

6.1 Assortment Simulator

We consider the following latent-class multinomial-logit (LC-MNL) model, in which each customer belongs to one of finitely many latent classes and, conditional on the class, chooses from the offered assortment according to an MNL model. This is a classical model for representing heterogeneity in discrete choice (Greene and Hensher, 2003).

The model has $n = 128$ products and $C = 32$ latent classes. The products are partitioned into C groups of four products, with group k associated with class k . We sample the product revenues $r_j \stackrel{\text{i.i.d.}}{\sim} \text{Uniform}(1, 10)$ once and hold them fixed throughout the experiment. A context $z \in \mathbb{R}^{16}$ determines the class probabilities:

$$\pi_k(z) = \frac{\exp(\theta_k^\top z)}{\sum_{\ell=1}^C \exp(\theta_\ell^\top z)}, \quad k = 1, \dots, C, \quad (6)$$

where $\theta_k \in \mathbb{R}^{16}$ is the vector of context coefficients for class k . We likewise sample the context-coefficient vectors $\theta_k \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}(0, I)$ once and hold them fixed throughout the experiment. The utility of product j for class k is $v_{kj} = 8$ if product j belongs to group k and $v_{kj} = -8$ otherwise. The outside option has utility zero. Thus, customers in class k strongly prefer the four products associated with their class, while products in the other groups are less attractive than the outside option.

An assortment is represented by $x \in \{0, 1\}^{128}$. Conditional on class k , a customer offered x

purchases product j with probability

$$P_k(j | x) = \frac{x_j \exp(v_{kj})}{1 + \sum_{\ell=1}^n x_\ell \exp(v_{k\ell})}, \quad (7)$$

and chooses the outside option with the remaining probability. The ground-truth expected revenue from offering assortment x to a customer with context z is therefore

$$R^*(x, z) = \sum_{k=1}^C \pi_k(z) \frac{\sum_{j=1}^n r_j x_j \exp(v_{kj})}{1 + \sum_{j=1}^n x_j \exp(v_{kj})}. \quad (8)$$

6.2 Data Generation

We generate a supervised dataset of observations (x, z, j, y) , where $x \in \{0, 1\}^{128}$ is the offered assortment, $z \in \mathbb{R}^{16}$ is the customer context, $j \in \{0, \dots, n\}$ is the chosen option (with $j = 0$ denoting the outside option), and y is the realized transaction revenue. We generate 1,000,000 training observations and 100,000 validation observations.

For each observation, we draw $z \sim \mathcal{N}(0, I)$ and construct an assortment x by including each product independently with probability $1/2$. We then draw a latent class according to (6), draw a choice according to (7), and record the realized revenue y . The realized revenue equals r_j if product j is purchased and zero if the outside option is chosen. The resulting training revenues are therefore noisy observations of the ground-truth expected revenue $R^*(x, z)$.

6.3 Models and Training Protocol

We compare DFN with several benchmark models. Some models predict revenue directly as a function of the assortment and context, while others model customer choice and compute predicted revenue from the resulting purchase probabilities. We optimize each trained model to select an assortment.

DFN. We use the practical DFN described in Section 3 to predict transaction revenue from the assortment and context. We consider configurations with 64, 128, and 256 nodes on each side of the bipartite graph. In every configuration, the cost network has one ReLU hidden layer of width 256. The networks defining $\alpha(z)$ and $\beta(z)$ each have one ReLU hidden layer of width 64. We initialize the output-layer biases of these networks so that their baseline values are $\alpha = -0.005$ and $\beta = -2$, which keeps the initial predictions on a suitable scale. We train W , h , and all parameters of the networks producing $c(z)$, $\alpha(z)$, and $\beta(z)$ jointly by minimizing MSE between predicted and realized transaction revenues using Adam with minibatches of 256, an initial learning rate of 0.003, no weight decay, cosine annealing, and 15 epochs. We select the configuration and training checkpoint with the lowest validation MSE.

MLP. The MLP is a fully connected neural network that takes the concatenated assortment and context (x, z) as input and outputs a scalar revenue prediction. We consider configurations with two ReLU hidden layers of equal width, using widths 256, 512, and 1024. We train all weights and biases of each configuration by minimizing MSE between predicted and realized transaction revenues using Adam with minibatches of 512, an initial learning rate of 0.001, weight decay 0.001, cosine annealing, and 150 epochs. We select the configuration and training checkpoint with the lowest validation MSE.

Contextual MNL. In the contextual multinomial-logit model, product j has utility $u_j(z) = a_j + w_j^\top z$, while the outside option has utility zero (Train, 2009). The probability that product j is purchased is

$$P(j \mid x, z) = \frac{x_j e^{u_j(z)}}{1 + \sum_{\ell=1}^n x_\ell e^{u_\ell(z)}}.$$

Thus, each product has one intercept and one 16-dimensional context coefficient. The utility of the outside option remains fixed at zero. We train (a_j, w_j) for $j = 1, \dots, n$ by minimizing the negative log-likelihood of the observed choices using Adam with minibatches of 512, an initial learning rate of 0.01, weight decay 0.001, cosine annealing, and 150 epochs. We retain the training checkpoint with the lowest validation negative log-likelihood.

Contextual LC-MNL. We fit the LC-MNL model used by the assortment simulator in Section 6.1 with 10 and 32 latent classes, denoted by LC-MNL-10 and LC-MNL-32. The simulator has 32 latent classes, so LC-MNL-10 is misspecified and LC-MNL-32 is correctly specified. For each model, we estimate the context-coefficient vector θ_k for every latent class k and the class-specific product utility v_{kj} for every product j , as defined in Equations (6) and (7), with $C = 10$ or $C = 32$. All outside-option utilities remain fixed at zero. We minimize the negative log-likelihood of the observed choices using Adam with minibatches of 512, an initial learning rate of 0.005, weight decay 10^{-4} , cosine annealing, and 150 epochs. For each model, we retain the training checkpoint with the lowest validation negative log-likelihood.

DeepMNL. Let e_j denote the one-hot identity of product j , with $e_0 = 0$ for the outside option. DeepMNL applies the same scalar-valued neural network to every product (Aouad and Désir, 2025):

$$u_j(z) = N_\theta(e_j, z), \quad j = 0, \dots, n.$$

The probability that product j is purchased is

$$P(j \mid x, z) = \frac{x_j e^{u_j(z)}}{e^{u_0(z)} + \sum_{\ell=1}^n x_\ell e^{u_\ell(z)}}.$$

Following the synthetic-experiment training settings of Aouad and Désir (2025), we use exponential linear unit (ELU) activations and consider a linear utility network, a utility network with one

hidden layer of width 3, and a utility network with two hidden layers of width 5. We train all weights and biases of each candidate utility network by minimizing the negative log-likelihood of the observed choices using Adam with minibatches of 32, an initial learning rate of 0.001, no weight decay, and at most 100 epochs. Training stops after ten consecutive epochs without an improvement in validation negative log-likelihood. We select the configuration and training checkpoint with the lowest validation negative log-likelihood.

RUMnet. RUMnet augments the product identity and context with learned latent representations (Aouad and Désir, 2025). For $a, b = 1, \dots, L$, let $E_a(e_j)$ and $H_b(z)$ be representations of product j and context z . The utility of product j in component (a, b) is

$$u_j^{ab}(z) = N_\theta(e_j, E_a(e_j), z, H_b(z)).$$

The corresponding purchase probability is

$$P_{ab}(j \mid x, z) = \frac{x_j e^{u_j^{ab}(z)}}{e^{u_0^{ab}(z)} + \sum_{\ell=1}^n x_\ell e^{u_\ell^{ab}(z)}}.$$

RUMnet averages these probabilities over the L^2 components:

$$P(j \mid x, z) = \frac{1}{L^2} \sum_{a=1}^L \sum_{b=1}^L P_{ab}(j \mid x, z).$$

Following the synthetic-experiment training settings of Aouad and Désir (2025), we use five-dimensional representations, ELU activations, $L \in \{2, 5\}$, and the same three utility-network architectures used for DeepMNL. We train each candidate by minimizing the negative log-likelihood of the observed choices with respect to all weights and biases of E_a , H_b , and N_θ . We use Adam with minibatches of 32, an initial learning rate of 0.001, no weight decay, and at most 100 epochs. Training stops after ten consecutive epochs without an improvement in validation negative log-likelihood. We select the configuration and training checkpoint with the lowest validation negative log-likelihood.

Res-Assort-Net(f). The feature-based Res-Assort-Net uses a product encoder E_ϕ and a context encoder H_ψ . Let e_j be the one-hot vector identifying product j . For assortment x , the encoders produce

$$\begin{aligned} q_0(z) &= E_\phi(0)^\top H_\psi(z), \\ q_j(x, z) &= E_\phi(x_j e_j)^\top H_\psi(z), \quad j = 1, \dots, n. \end{aligned}$$

Let $q(x, z) = (q_0(z), q_1(x, z), \dots, q_n(x, z))$. We concatenate $q(x, z)$ with $(1, x)$, where the leading 1 indicates that the outside option is always available. A residual network S_ω maps the resulting vector to one score per option. Writing $s(x, z) = (s_0(x, z), \dots, s_n(x, z))$, we have

$$s(x, z) = S_\omega(q(x, z), 1, x).$$

The model predicts the purchase probability

$$P(j \mid x, z) = \frac{x_j e^{s_j(x, z)}}{e^{s_0(x, z)} + \sum_{\ell=1}^n x_\ell e^{s_\ell(x, z)}}.$$

Following Wang et al. (2023), we consider three configurations. In the first two, the context encoder has two layers of widths 100 and 10, and the product encoder has two layers of width 10. In the third, the context encoder has two layers of widths 200 and 20, and the product encoder has two layers of width 20. The three configurations use, respectively, one single-layer residual block, one residual block with hidden width 60, and two residual blocks with hidden width 60. We train all weights and biases of E_ϕ , H_ψ , and S_ω by minimizing the negative log-likelihood of the observed choices using Adam with minibatches of 100, an initial learning rate of 0.0005, no weight decay, and 100 epochs. We select the configuration and training checkpoint with the lowest validation negative log-likelihood.

MCCM. The Markov chain choice model represents each product and the outside option as a state (Blanchet et al., 2016). A customer begins in a state according to an arrival distribution and transitions between unavailable products until reaching an offered product or the absorbing outside option. Following Şimşek and Topaloglu (2018), we train one context-blind MCCM on the pooled observations by estimating its arrival distribution and product-transition probabilities with their expectation-maximization algorithm. We initialize these probabilities uniformly, keep the outside option absorbing, and stop when the observed-data log-likelihood improves by less than 0.01% in two successive iterations.

6.4 Downstream Optimization

We draw 100 contexts independently from $\mathcal{N}(0, I)$. For each context, we maximize the predicted revenue of every trained model over the common feasible set

$$\mathcal{X} = \left\{ x \in \{0, 1\}^{128} : \mathbf{1}^\top x \leq 16 \right\}. \quad (9)$$

DFN. For a fixed context, maximizing the trained DFN over $\mathcal{X}$ is equivalent to a MILP. The formulation combines the assortment constraints with the transportation problem defining the DFN. The complete formulation appears in Equation (B.1) in the Appendix.

MLP. For a fixed context, maximizing the trained MLP over $\mathcal{X}$ is equivalent to a MILP. We encode each ReLU neuron using the standard exact big- M formulation (Tjeng et al., 2019). The complete formulation and the construction of its bounds appear in Appendix B.1.5.

Choice Models. For a fixed context, the contextual MNL, LC-MNL, DeepMNL, and RUMnet models are optimized using an exact mixed-MNL linearization (Méndez-Díaz et al., 2014). We

optimize MCCM using an exact cardinality-constrained formulation (Désir et al., 2020). For Res-Assort-Net(f), we encode the ReLU network exactly and represent its softmax purchase probabilities using Gurobi’s nonlinear constraints.

Reference Policies. We report three reference policies. The oracle maximizes the true 32-class contextual LC-MNL model over $\mathcal{X}$ using the exact mixed-MNL formulation. The top-revenue policy offers the 16 products with the largest revenues. The random-capacity policy samples one assortment of 16 products uniformly for each decision context.

6.5 Software and Hardware

We use Python 3.13.5, PyTorch 2.8.0, NumPy 2.3.4, Python Optimal Transport 0.9.6.post1, and Gurobi 12.0.3 on a MacBook Pro with an Apple M4 Max chip, 14 CPU cores, and 36 GB of RAM.

6.6 Results

Table 3 reports the mean true expected revenue of the incumbent assortments.

Table 3: Mean True Expected Revenue of Incumbent Assortments at Each Solver-Time Checkpoint

Model	1 s	5 s	10 s	30 s	60 s
Oracle	8.787				
DFN	8.080	8.158	8.161	8.157	8.157
MLP	1.345 ^a	1.945 ^b	2.393 ^c	4.955	6.613
Contextual MNL	3.433	3.433	3.433	3.433	3.433
LC-MNL-10	4.000	4.670	4.668	4.675	4.675
LC-MNL-32	3.804	4.087	4.486	4.639	4.898
DeepMNL	6.396	6.396	6.396	6.396	6.396
RUMnet	4.477	7.070	7.208	7.140	7.120
MCCM	6.419	6.412	6.412	6.412	6.412
Res-Assort-Net(f)	4.120 ^d	4.061	4.061	4.061	4.061
Top-revenue	3.843	3.843	3.843	3.843	3.843
Random capacity	2.906	2.906	2.906	2.906	2.906

Note. Results are averaged over 100 decision contexts, except where a solver had not yet found an incumbent: ^a90 contexts; ^b98 contexts; ^c99 contexts; and ^d98 contexts.

DFN produces the highest-revenue assortments among the learned models at every checkpoint. Its mean true revenue is already 8.080 after one second and reaches 8.157 at 60 seconds, or 92.8% of the oracle revenue of 8.787. At 60 seconds, MLP attains 6.613. Among the choice models, RUMnet attains 7.120, MCCM 6.412, DeepMNL 6.396, LC-MNL-32 4.898, LC-MNL-10 4.675, and contextual MNL 3.433. Res-Assort-Net(f) attains 4.061. DFN also exceeds the top-revenue and random-capacity reference policies, which attain 3.843 and 2.906, respectively.

The MLP formulation yields low-revenue incumbents at short time limits, reaching 1.345 after one second, and remains below DFN after 60 seconds. Among the choice models, RUMnet provides the strongest final decision quality, while MCCM provides the strongest one-second result. Despite

attaining the lowest validation negative log-likelihood among the choice models, Res-Assort-Net(f) attains lower true revenue after optimization than DeepMNL, RUMnet, and MCCM.

Overall, DFN combines efficient downstream optimization with strong decision performance, producing the highest-revenue assortments among the learned models throughout the optimization horizon. This combination is particularly important in applications that require repeated decisions under short time limits. The personalized online assortment problem considered in this section is one such application. For example, an online platform offering personalized assortments must select an assortment in real time for each arriving customer based on their context. In such applications, DFN’s ability to produce high-revenue assortments within the available time can be particularly valuable.

7 Additional Model Diagnostics

We conduct three additional experiments to better understand the practical behavior of DFNs. First, the initialization of A is intended to keep $\Delta(A)$ small and facilitate downstream optimization. We compare a learned A with one fixed at its initialization to measure the resulting tradeoff between predictive accuracy and solve time. Second, our theory links the complexity of optimizing a DFN to $\Delta(A)$. We therefore track $\Delta(A)$ and solve time throughout training to examine whether larger values of $\Delta(A)$ are also associated with longer solve times in practice. Finally, while the neural-network parameters can use standard initialization schemes, the integer parameters require a separate initialization choice. We test the sensitivity of DFN to this choice. We conduct all three experiments on the superconductor composition-design task from Section 4, in which $x \in \mathbb{Z}_+^{77}$ encodes the composition of a material across 77 chemical elements, the context z gives the number of constituent elements and their total coefficient, and the outcome is the critical temperature. We select this task because A moves substantially during training and its downstream solve times are sufficiently large for changes in optimization difficulty to be measured clearly.

7.1 Learning Versus Fixing the Balance Matrix

Description. The initialization of A is intended to keep $\Delta(A)$ small and facilitate downstream optimization, but keeping A at this initialization may limit its approximation power. We examine this tradeoff by comparing a DFN that learns A with a fixed- A variant. In the latter, W remains fixed at its near-identity initialization, and hence $A = [W^\top, -W^\top]^\top$ also remains fixed, while h , the cost network, and the output-scaling networks are trained normally. The experiment measures how much predictive accuracy is gained by learning A and how this gain affects downstream solve time.

Experimental Setup. For each of the five training seeds, the fixed- A model uses the same architecture and training hyperparameters as the corresponding DFN with learned A . Both variants use the superconductor data split, preprocessing, and training protocol from Section 4. We report standardized test MSE over the five seeds. For downstream optimization, we use the seed-zero

models and the same five contexts, feasible sets, and exact MILP formulation as in Section 5. Each solve has a 15-minute time limit.

Results. Table 4 reports the results.

Table 4: Effect of Fixing the Balance Matrix on the Superconductor Task

Model	Standardized test MSE	Mean solve time (s)
DFN	0.0996 ± 0.0011	5.940 ± 2.535
DFN with fixed A	0.1163 ± 0.0046	0.215 ± 0.051

Notes. Test MSE is reported as mean $\pm$ standard error over five training seeds. Solve time is reported as mean $\pm$ standard error over five test contexts for the seed-zero models.

Learning A reduces standardized test MSE from 0.1163 to 0.0996. Nevertheless, the fixed- A model remains a strong predictor: its MSE is slightly lower than the MLP value of 0.1176 ± 0.0113 reported in Section 4.4. Fixing A also reduces mean downstream solve time from 5.940 seconds to 0.215 seconds. Thus, fixing A sacrifices some predictive accuracy but makes the DFN even easier to optimize.

7.2 Evolution of A During Training

Description. The theory developed in Section 2.3 gives a complexity bound that depends on $\Delta(A)$, and the initialization of A is intended to keep this parameter small. We examine whether $\Delta(A)$ is correlated with downstream MILP solve time in practice by tracking how $\Delta(A)$ and solve time evolve as A moves away from its initialization A_0 during training.

Experimental Setup. We use the architecture and training hyperparameters of the seed-zero superconductor DFN from Section 4 and repeat the experiment under three random seeds. We train each model for 300 epochs without early stopping and save W at initialization and every 10 epochs. To isolate the effect of A , we take the model at the end of training, hold all parameters other than W fixed, and replace W by its value at each checkpoint. We then solve the downstream problem for the five test contexts used in Section 5, with a 15-minute time limit for each solve. At each checkpoint, we report $\|A - A_0\|_1$, $\log \Delta(A)$, and mean solve time. The curves show the mean and standard error over the three training seeds; for each seed, solve time is first averaged over the five contexts.

Results. Figure 1 shows the results. Over 300 epochs, both $\|A - A_0\|_1$ and $\log \Delta(A)$ increase substantially. Mean solve time rises from 0.539 ± 0.128 seconds at initialization to 11.039 ± 0.665 seconds after 300 epochs. Across the 31 checkpoint averages, solve time has Pearson correlations of 0.72 with $\|A - A_0\|_1$ and 0.73 with $\log \Delta(A)$. These results show that the learned matrix moves substantially away from its initialization and that, in this experiment, larger values of $\Delta(A)$ are associated with longer MILP solve times.

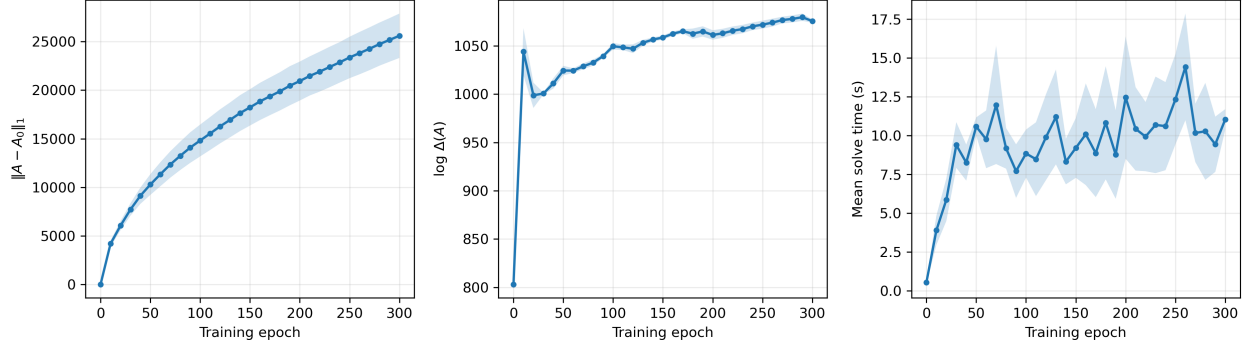


Figure 1: Evolution of the Balance Matrix and Downstream Optimization Time on the Superconductor Task. Shaded regions show one standard error over three training seeds.

7.3 Sensitivity to Integer-Parameter Initialization

Description. The initialization of A is intended to facilitate downstream optimization, but learning should not depend critically on one particular initialization. We therefore examine how the initialization of (W, h) affects predictive performance.

Experimental Setup. For each of the five training seeds, all four initialization schemes use the architecture and training hyperparameters of the corresponding superconductor DFN from Section 4. We compare the near-identity initialization of W with $h = \mathbf{0}$ against three alternatives: the entries of W are sampled independently and uniformly from $\{0, \dots, 5\}$ while $h = \mathbf{0}$; W uses the near-identity initialization while the entries of h are sampled independently and uniformly from $\{0, \dots, 5\}$; and the entries of both W and h are sampled independently and uniformly from $\{0, \dots, 5\}$. All other aspects of training are held fixed. We report standardized test MSE as mean $\pm$ standard error over the five seeds.

Results. Table 5 reports the results. The mean standardized test MSE ranges from 0.0977 to 0.1058 across the four initialization schemes. Thus, predictive performance remains broadly stable across the initializations of both W and h considered in this experiment.

Table 5: Sensitivity of Predictive Performance to the Initialization of the Integer Parameters

Initialization of W	Initialization of h	Standardized test MSE
Near identity	Zero	0.0996 ± 0.0011
Uniform on $\{0, \dots, 5\}$	Zero	0.1058 ± 0.0034
Near identity	Uniform on $\{0, \dots, 5\}$	0.0977 ± 0.0016
Uniform on $\{0, \dots, 5\}$	Uniform on $\{0, \dots, 5\}$	0.1027 ± 0.0026

8 Conclusion and Future Directions

We introduced Deep Flow Networks (DFNs), a class of function approximators for predict-then-optimize problems with integer decisions. We established a universality result for DFNs and connected the complexity of optimizing a trained DFN to its deviation from the M -convex regime through $\Delta(A)$. We developed a practical contextual implementation for learning from data and tested its predictive performance and downstream optimization speed across 31 prediction tasks constructed from public datasets. We found that it achieves competitive predictive performance relative to MLPs while being substantially easier to optimize. We also conducted an end-to-end predict-then-optimize assortment experiment and found that DFNs produce the highest-revenue assortments among the learned models and do so quickly.

DFNs take longer to train than MLPs because their training requires differentiation through a transportation optimization layer. Although optimization layers are standard in differentiable optimization (Amos and Kolter, 2017; Agrawal et al., 2019), they are more computationally expensive than conventional neural-network layers. Reducing DFN training time remains an important direction for future work. Promising directions include developing GPU implementations for the transportation problems used during training and exploring DFN architectures based on transportation problems that admit faster solution methods, such as transportation on a line with distance-based costs, whose optimal value can be computed directly from cumulative node balances. That said, the additional training cost can be amortized in applications where a model is trained offline and then optimized repeatedly across many contexts or feasible sets.

Another direction is to extend the theory and practical implementations to broader classes of discrete ground-truth functions beyond minimum-cost-flow value functions. A further direction is to use structured function approximators such as DFNs to represent action-value functions in reinforcement-learning problems with large discrete action spaces, where selecting an action requires repeatedly optimizing the learned function.

References

- Akshay Agrawal, Brandon Amos, Shane Barratt, Stephen Boyd, Steven Diamond, and J. Zico Kolter. Differentiable convex optimization layers. In *Advances in Neural Information Processing Systems*, volume 32, 2019.
- Ravindra K. Ahuja, Thomas L. Magnanti, and James B. Orlin. *Network Flows: Theory, Algorithms, and Applications*. Prentice Hall, Englewood Cliffs, NJ, 1993. ISBN 9780136175490.
- Brandon Amos and J. Zico Kolter. OptNet: Differentiable optimization as a layer in neural networks. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 136–145, 2017.
- Brandon Amos, Lei Xu, and J. Zico Kolter. Input convex neural networks. In *International Conference*

- on *Machine Learning (ICML)*, volume 70 of *Proceedings of Machine Learning Research*, pages 146–155, 2017.
- Analytics Vidhya. Food demand forecasting. DataHack practice problem, 2019. URL <https://www.analyticsvidhya.com/datahack/contest/genpact-machine-learning-hackathon-1/>.
- Tomasz Antczak and Rafał Weron. Point of sale (POS) data from a supermarket: Transactions and cashier operations. *Data*, 4(2):67, 2019. doi: 10.3390/data4020067.
- Ali Aouad and Antoine Désir. Representing random utility choice models with neural networks. *Management Science*, 72(8):6686–6701, 2025. doi: 10.1287/mnsc.2023.02189.
- Imre Bárány. On the power of linear dependencies. In *Building Bridges: Between Mathematics and Computer Science*, pages 31–45. Springer, 2008.
- Yoshua Bengio, Nicholas Léonard, and Aaron Courville. Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv preprint arXiv:1308.3432*, 2013.
- Omar Besbes and Assaf J. Zeevi. Dynamic pricing without knowing the demand function: Risk bounds and near-optimal algorithms. *Operations Research*, 57(6):1407–1420, 2009.
- Jose Blanchet, Guillermo Gallego, and Vineet Goyal. A markov chain approximation to choice modeling. *Operations Research*, 64(4):886–905, 2016. doi: 10.1287/opre.2016.1505.
- Brad Boehmke and Steven M. Mortimer. completejourney: Retail shopping data. R package, 2019. URL <https://bradleyboehmke.github.io/completejourney/>.
- Giuseppe C. Calafiore, Stéphane Gaubert, and Corrado Possieri. Log-sum-exp neural networks and posynomial models for convex and log-log-convex data. *IEEE Transactions on Neural Networks and Learning Systems*, 31(3):827–838, 2020. doi: 10.1109/TNNLS.2019.2910417.
- Ozan Candogan and Ayoub Foussoul. Deep Flow Networks. In *Proceedings of the 43rd International Conference on Machine Learning*, 2026.
- Centers for Medicare & Medicaid Services. Payroll based journal daily nurse staffing and provider information data. CMS Provider Data Catalog, 2026. URL <https://data.cms.gov/quality-of-care/payroll-based-journal-daily-nurse-staffing>.
- Charles Cernay, Elise Pelzer, and David Makowski. Data from: A global experimental dataset for assessing grain legume production, 2017.
- Daqing Chen. Online retail II, 2012.
- City and County of San Francisco. Fire department calls for service. DataSF, 2026. URL <https://data.sfgov.org/Public-Safety/Fire-Department-Calls-for-Service/nuek-vuh3>.

- CityBikes contributors. Bike-share data. Historical bike-share data archive, 2025. URL <https://data.citybik.es/>.
- Maxime C. Cohen, Paul-Emile Gras, Arthur Pentecoste, and Renyu Zhang. *Demand Prediction in Retail: A Practical Guide to Leverage Data and Predictive Analytics*, volume 14 of *Springer Series in Supply Chain Management*. Springer, 2022. doi: 10.1007/978-3-030-85855-1.
- Corporación Favorita, inversion, Julia Elliott, and Mark McDonald. Corporación favorita grocery sales forecasting. Kaggle, 2017. URL <https://www.kaggle.com/competitions/favorita-grocery-sales-forecasting>.
- Antoine Désir, Vineet Goyal, Danny Segev, and Chun Ye. Constrained assortment optimization under the markov chain-based choice model. *Management Science*, 66(2):698–721, 2020. doi: 10.1287/mnsc.2018.3230.
- Eustache Diemert, Julien Meynet, Pierre Galland, and Damien Lefortier. Attribution modeling increases efficiency of bidding in display advertising. In *Proceedings of the AdKDD and TargetAd Workshop at KDD*, 2017. URL <https://ailab.criteo.com/criteo-attribution-modeling-bidding-dataset/>.
- Brian W Dolhansky and Jeff A Bilmes. Deep submodular functions: Definitions and learning. *Advances in Neural Information Processing Systems*, 29, 2016.
- Priya L. Donti, Brandon Amos, and J. Zico Kolter. Task-based end-to-end model learning in stochastic optimization. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- Adam N. Elmachtoub and Paul Grigas. Smart “predict, then optimize”. *Management Science*, 68(1):9–26, 2022. doi: 10.1287/mnsc.2020.3922.
- Pierluigi Favati and Fiorella Tardella. Convexity in nonlinear integer programming. *Ricerca Operativa*, 53:3–44, 1990. ISSN 0390-8127.
- Aaron Ferber, Bryan Wilder, Bistra Dilkina, and Milind Tambe. MIPaaL: Mixed integer program as a layer. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pages 1504–1511, 2020. doi: 10.1609/aaai.v34i02.5509.
- Steven D. Galbraith. *Mathematics of Public Key Cryptography*. Cambridge University Press, 2012. ISBN 9781107013926.
- Chongming Gao, Shijun Li, Wenqiang Lei, Jiawei Chen, Biao Li, Peng Jiang, Xiangnan He, Jiaxin Mao, and Tat-Seng Chua. KuaiRec: A fully-observed dataset and insights for evaluating recommender systems. In *Proceedings of the 31st ACM International Conference on Information and Knowledge Management*, pages 540–550, 2022. doi: 10.1145/3511808.3557220.

- Beatrice Gehri, Rene Schwendimann, Stefanie Bachnick, Michael Ketzer, and Michael Simon. Matching registered nurse services with changing care demands in psychiatric hospitals: Data from the nurse survey, 2023.
- William H. Greene and David A. Hensher. A latent class model for discrete choice analysis: Contrasts with mixed logit. *Transportation Research Part B: Methodological*, 37(8):681–698, 2003. doi: 10.1016/S0191-2615(02)00046-2.
- Bjarne Grimstad and Henrik Andersson. Relu networks as surrogate models in mixed-integer linear programs. *Computers & Chemical Engineering*, 131:106580, 2019. doi: 10.1016/j.compchemeng.2019.106580.
- Peter Gritzmann and Victor Klee. On the 0–1-maximization of positive definite quadratic forms. In *Operations Research Proceedings 1988*, pages 222–227. Springer, Berlin, Heidelberg, 1989. doi: 10.1007/978-3-642-74862-2_61.
- Hans-Martin Gutmann. A radial basis function method for global optimization. *Journal of Global Optimization*, 19(3):201–227, 2001. doi: 10.1023/A:1011255519438.
- Pavithra Harsha, Ashish Jagmohan, Jayant Kalagnanam, Brian Quanz, and Divya Singhvi. Deep policy iteration with integer programming for inventory management. *Manufacturing & Service Operations Management*, 27(2):369–388, 2025. doi: 10.1287/msom.2022.0617.
- Matthias Hofmann and Turid Siebenbrunner. A rich dataset of hourly residential electricity consumption data and survey answers from the iFlex dynamic pricing experiment. *Data in Brief*, 50:109571, 2023. doi: 10.1016/j.dib.2023.109571.
- Roger A. Horn and Charles R. Johnson. *Matrix Analysis*. Cambridge University Press, 2 edition, 2012.
- Seyed Mohammad Hosseini, Arash Jamshidi, Seyed Mahdi Noormousavi, Mahdi Siavoshani, and Naeimeh Omidvar. Extended deep submodular functions. *Transactions on Machine Learning Research*, 2024. ISSN 2835-8856. URL <https://openreview.net/forum?id=qoxusn9u2L>.
- James M. Kilts Center for Marketing. Dominick’s dataset. University of Chicago Booth School of Business, 2026. URL <https://www.chicagobooth.edu/research/kilts/research-data/dominicks>.
- Donald R. Jones, Matthias Schonlau, and William J. Welch. Efficient global optimization of expensive black-box functions. *Journal of Global Optimization*, 13(4):455–492, 1998.
- Waquar Kaleem, Doyoung Lee, Changhyun Kwon, and Anirudh Subramanyam. Neural embedded mixed-integer optimization for location-routing problems. In *Workshop on Differentiable Learning of Combinatorial Algorithms*, 2025. URL <https://openreview.net/forum?id=WCILzWMXq9>.

- Heysem Kaya, Pinar Tufekci, and Erdinc Uzun. Gas turbine CO and NOx emission data set, 2019.
- Alyssa Kody, Samuel Chevalier, Spyros Chatzivasileiadis, and Daniel Molzahn. Modeling the AC power flow equations with optimally compact neural networks: Application to unit commitment. *Electric Power Systems Research*, 213:108282, 2022. doi: 10.1016/j.epsr.2022.108282.
- Anna Konovalenko, Lars Magnus Hvattum, and Kim Aleksander Hammer Iversen. Data describing planned routes and driven routes from a last-mile delivery vehicle routing problem. *Data in Brief*, 66:112744, 2026. doi: 10.1016/j.dib.2026.112744.
- Max Kuhn and Kjell Johnson. *Applied Predictive Modeling*. Springer, New York, 2013. doi: 10.1007/978-1-4614-6849-3.
- Jakub Kuzilek, Martin Hlosta, and Zdenek Zdrahal. Open university learning analytics dataset, 2015.
- Jeffrey C. Lagarias, Jr. Lenstra, Hendrik W., and Claus-Peter Schnorr. Korkin–zolotarev bases and successive minima of a lattice and its reciprocal lattice. *Combinatorica*, 10(4):333–348, 1990. doi: 10.1007/BF02128669.
- László Lovász. Submodular functions and convexity. In Achim Bachem, Bernhard Korte, and Martin Grötschel, editors, *Mathematical Programming: The State of the Art (Bonn 1982)*, pages 235–257. Springer, Berlin, Heidelberg, 1983. doi: 10.1007/978-3-642-68874-4_10.
- Alessandro Magnani and Stephen P. Boyd. Convex piecewise-linear fitting. *Optimization and Engineering*, 10(1):1–17, 2009. doi: 10.1007/s11081-008-9045-3.
- Spyros Makridakis, Evangelos Spiliotis, and Vassilios Assimakopoulos. The M5 accuracy competition: Results, findings, and conclusions. *International Journal of Forecasting*, 38(4):1346–1364, 2022. doi: 10.1016/j.ijforecast.2021.11.013.
- Jayanta Mandi, James Kotary, Senne Berden, Maxime Mulamba, Víctor Bucarey, Tias Guns, and Ferdinando Fioretto. Decision-focused learning: Foundations, state of the art, benchmark and future opportunities. *Journal of Artificial Intelligence Research*, 80:1623–1701, 2024. doi: 10.1613/jair.1.15320.
- Isabel Méndez-Díaz, Juan José Miranda-Bront, Gustavo Vulcano, and Paula Zabala. A branch-and-cut algorithm for the latent-class logit assortment problem. *Discrete Applied Mathematics*, 164(1):246–263, 2014. doi: 10.1016/j.dam.2012.03.003.
- Microsoft Research. MIND: Microsoft News Dataset. Public research dataset, 2020. URL <https://msnews.github.io/>.
- Velibor V. Mišić. Optimization of tree ensembles. *Operations Research*, 68(5):1605–1624, 2020. doi: 10.1287/opre.2019.1928.

- Sérgio Moro, Paulo Rita, and Paulo Cortez. Bank marketing, 2014.
- Kazuo Murota. *Discrete Convex Analysis*, volume 10 of *SIAM Monographs on Discrete Mathematics and Applications*. Society for Industrial and Applied Mathematics (SIAM), Philadelphia, PA, 2003. ISBN 9780898715408. doi: 10.1137/1.9780898718508.
- New York City Taxi and Limousine Commission. Tlc trip record data, 2024. URL <https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page>.
- Olist. Brazilian e-commerce public dataset by olist. Kaggle, 2018. URL <https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce>.
- James Oxley. *Matroid Theory*. Oxford University Press, second edition, 2011.
- Posit Software, PBC. Deliveries data. modeldata package documentation, 2026. URL <https://modeldata.tidymodels.org/reference/deliveries.html>.
- Rossmann. Rossmann store sales. Kaggle, 2015. URL <https://www.kaggle.com/c/rossmann-store-sales>.
- Tobias Rothvoss. Lattices (lecture notes). <https://sites.math.washington.edu/~rothvoss/599-winter-2023/lattices.pdf>, 2023.
- San Francisco Municipal Transportation Agency. SFpark evaluation data, 2014. URL <https://www.sfmta.com/getting-around/drive-park/demand-responsive-pricing/sfpark-evaluation>.
- Alexander Schrijver. *Theory of Linear and Integer Programming*. Wiley-Interscience Series in Discrete Mathematics and Optimization. John Wiley & Sons, Chichester and New York, 1986. ISBN 0471908541.
- Artur M. Schweidtmann and Alexander Mitsos. Deterministic global optimization with artificial neural networks embedded. *Journal of Optimization Theory and Applications*, 180(3):925–948, 2019. doi: 10.1007/s10957-018-1396-0.
- Akiyoshi Shioura. M-convex function minimization under L1-distance constraint and its application to dock reallocation in bike-sharing system. *Mathematics of Operations Research*, 47(2):1566–1611, 2022. doi: 10.1287/moor.2021.1180.
- A. Serdar Şimşek and Huseyin Topaloglu. Technical note—an expectation-maximization algorithm to estimate the parameters of the markov chain choice model. *Operations Research*, 66(3):748–760, 2018. doi: 10.1287/opre.2017.1692.
- Jimmy Smith. Retail transactions and stocks data. Mendeley Data, Version 1, 2026.
- Beata Strack, Jonathan P. DeShazo, Chris Gennings, Juan L. Olmo, Sebastian Ventura, Krzysztof J. Cios, and John N. Clore. Diabetes 130-US hospitals for years 1999–2008, 2014.

- Kalyan T. Talluri and Garrett J. Van Ryzin. *The Theory and Practice of Revenue Management*, volume 68 of *International Series in Operations Research & Management Science*. Springer, New York, NY, 2004. ISBN 978-1-4020-7701-2. doi: 10.1007/b139000.
- Éva Tardos. A strongly polynomial minimum cost circulation algorithm. *Combinatorica*, 5:247–255, 1985. doi: 10.1007/BF02579369.
- Vincent Tjeng, Kai Xiao, and Russ Tedrake. Evaluating robustness of neural networks with mixed integer programming. In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=HyGIIdiRqtm>.
- Brandon Trabucco, Aviral Kumar, Xinyang Geng, and Sergey Levine. Conservative objective models for effective offline model-based optimization. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 10358–10368. PMLR, 2021.
- Kenneth E. Train. *Discrete Choice Methods with Simulation*. Cambridge University Press, 2 edition, 2009.
- UCI Machine Learning Repository. Superconductivity data, 2018. URL <https://archive.ics.uci.edu/dataset/464/superconductivity+data>.
- UCI Machine Learning Repository. Productivity prediction of garment employees, 2020. URL <https://archive.ics.uci.edu/dataset/597/productivity+prediction+of+garment+employees>.
- U.S. Bureau of Transportation Statistics. Transtats: Airline on-time performance data. Public data portal, 2023. URL <https://www.transtats.bts.gov/>.
- Boudewijn van Dongen. BPI Challenge 2019, 2019.
- Marin Vlastelica, Anselm Paulus, Vít Musil, Georg Martius, and Michal Rolínek. Differentiation of blackbox combinatorial solvers. In *International Conference on Learning Representations*, 2020.
- Walmart. Walmart recruiting: Store sales forecasting. Kaggle, 2014. URL <https://www.kaggle.com/c/walmart-recruiting-store-sales-forecasting>.
- Hanzhao Wang, Zhongze Cai, Xiaocheng Li, and Kalyan Talluri. A neural network based choice model for assortment optimization. *arXiv preprint arXiv:2308.05617*, 2023. doi: 10.48550/arXiv.2308.05617.
- Bryan Wilder, Bistra Dilkina, and Milind Tambe. Melding the data-decisions pipeline: Decision-focused learning for combinatorial optimization. In *AAAI Conference on Artificial Intelligence (AAAI)*, 2019.

- Lixia Wu, Haomin Wen, Haoyuan Hu, Xiaowei Mao, Yutong Xia, Ergang Shan, Jianbin Zheng, Junhong Lou, Yuxuan Liang, Liuqing Yang, Roger Zimmermann, Youfang Lin, and Huaiyu Wan. LaDe: The first comprehensive last-mile delivery dataset from industry. *arXiv preprint arXiv:2306.10675*, 2023. doi: 10.48550/arXiv.2306.10675.
- I-Cheng Yeh. Concrete compressive strength, 2007.
- Ron Yurko. NBA RAPM datasets. SCORE Sports Data Repository, 2024. URL <https://data.scorenetwork.org/basketball/nba-rapm-data.html>.
- Manzil Zaheer, Satwik Kottur, Siamak Ravanbakhsh, Barnabas Poczos, Russ R Salakhutdinov, and Alexander J Smola. Deep sets. *Advances in neural information processing systems*, 30, 2017.
- Lin Zhao, Lefei Li, and Zuo-Jun Max Shen. Transactional and in-store display data of a large supermarket for data-driven decision-making. *Naval Research Logistics*, 67(8):617–626, 2020. doi: 10.1002/nav.21957.

Appendices

A Proofs

A.1 Proof of Theorem 1

Fix an arbitrary context $z \in \mathbb{R}^q$ and write $g(x) := g(x, z)$. For this fixed context, we construct a graph G , a node subset S , cost and capacity vectors c and u , an integer matrix A , an integer vector b , and scalars α and β . For each $x \in I$, let $f(x)$ denote the optimal value of the minimum-cost flow problem on G with costs c , capacities u , net outflows $Ax + b$ at the nodes in S , and zero net outflow at every other node. We construct these objects such that

$$|g(x) - (\alpha f(x) + \beta)| < \epsilon \quad \text{for every } x \in I.$$

Applying the construction at every z , with the same graph G and node subset S , defines the mappings $c(\cdot)$, $u(\cdot)$, $A(\cdot)$, and $b(\cdot)$ and the functions $\alpha(\cdot)$ and $\beta(\cdot)$, proving the theorem.

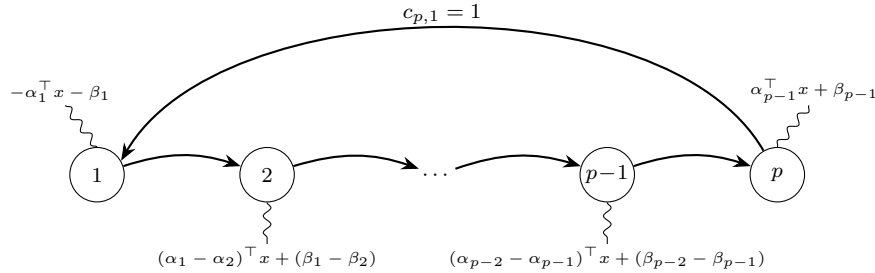


Figure A.1: Minimum-Cost Flow Construction for a Maximum of Affine Functions

Note. The construction represents $\max\{0, \max_{i=1, \dots, p-1} (\alpha_i^\top x + \beta_i)\}$. Arc $(p, 1)$ has cost 1, and the unlabeled arcs have cost 0. Every arc has capacity B , where $B := 2 \max_{x \in I} \max_{i=1, \dots, p-1} |\alpha_i^\top x + \beta_i|$. All nodes are special nodes $S = \{1, \dots, p\}$ with net outflows $-\alpha_1^\top x - \beta_1, (\alpha_1 - \alpha_2)^\top x + (\beta_1 - \beta_2), \dots, (\alpha_{p-2} - \alpha_{p-1})^\top x + (\beta_{p-2} - \beta_{p-1})$, and $\alpha_{p-1}^\top x + \beta_{p-1}$, respectively.

Let $\alpha_1, \dots, \alpha_{p-1} \in \mathbb{Z}^d$, $\beta_1, \dots, \beta_{p-1} \in \mathbb{Z}$, and let $I \subset \mathbb{Z}^d$ be finite. Set $B := 2 \max_{x \in I} \max_{i=1, \dots, p-1} |\alpha_i^\top x + \beta_i|$. We first show that the minimum-cost flow problem defined in Figure A.1 satisfies $f(x) = \max\{0, \max_{i=1, \dots, p-1} (\alpha_i^\top x + \beta_i)\}$ for every $x \in I$.

Consider the construction in Figure A.1. Fix $x \in I$, and let ξ be any feasible solution of (4). For every $i = 2, \dots, p$, define $I_i := \{i, \dots, p\}$. By construction, the only arc leaving I_i is $(p, 1)$ and the only arc entering I_i is $(i-1, i)$. Summing the net outflow constraints (4a) over all nodes in I_i yields

$$\begin{aligned} \xi_{p,1} - \xi_{i-1,i} &= \sum_{j=i}^{p-1} \left((\alpha_{j-1} - \alpha_j)^\top x + (\beta_{j-1} - \beta_j) \right) \\ &\quad + \alpha_{p-1}^\top x + \beta_{p-1} \\ &= \alpha_{i-1}^\top x + \beta_{i-1}. \end{aligned}$$

Since $\xi_{i-1,i} \geq 0$, we obtain $\xi_{p,1} \geq \alpha_{i-1}^\top x + \beta_{i-1}$ for all $i = 2, \dots, p$, i.e., $\xi_{p,1} \geq \max_{i=1, \dots, p-1} (\alpha_i^\top x + \beta_i)$. Moreover $\xi_{p,1} \geq 0$. Because the unique positive-cost arc is $(p, 1)$ with $c_{p,1} = 1$ and all other arcs have zero cost, we have $c^\top \xi = \xi_{p,1}$ and therefore

$$f(x) \geq \max \left\{ 0, \max_{i=1, \dots, p-1} (\alpha_i^\top x + \beta_i) \right\}.$$

For the reverse inequality, define $M := \max \left\{ 0, \max_{i=1, \dots, p-1} (\alpha_i^\top x + \beta_i) \right\}$. Set $\xi_{p,1} := M$ and, for each $i = 2, \dots, p$, set $\xi_{i-1,i} := M - (\alpha_{i-1}^\top x + \beta_{i-1}) \geq 0$. By the definition of B , $M \leq B/2$ and $\xi_{i-1,i} \leq B$. Thus, the constraints (4a)–(4c) are all satisfied, hence ξ is feasible and has objective value $c^\top \xi = \xi_{p,1} = M$. Therefore $f(x) \leq \max \left\{ 0, \max_{i=1, \dots, p-1} (\alpha_i^\top x + \beta_i) \right\}$.

Next, write $I = \{x^1, \dots, x^N\}$. For each i , pick any subgradient $s^i \in \partial g(x^i)$. Define the supporting affine functions ℓ_i , $i = 1, \dots, N$, by

$$\begin{aligned} \ell_i(x) &:= g(x^i) + (s^i)^\top (x - x^i) \\ &= (s^i)^\top x \\ &\quad + \left(g(x^i) - (s^i)^\top x^i \right). \end{aligned}$$

By convexity, $\ell_i(x) \leq g(x)$ for all $x \in \mathbb{R}^d$, and $\ell_i(x^i) = g(x^i)$. Let $h(x) := \max_{i=1, \dots, N} \ell_i(x)$. Then $h(x) \leq g(x)$ for all x , and for each $i \in \{1, \dots, N\}$, $h(x^i) \geq \ell_i(x^i) = g(x^i)$, while $h(x^i) \leq g(x^i)$, hence $h(x) = g(x)$ for every $x \in I$.

Let $m_I := \min_{x \in I} g(x)$. Note that $g(x) - m_I \geq 0$. Hence, for every $x \in I$,

$$\begin{aligned} g(x) - m_I &= h(x) - m_I \\ &= \max \{ 0, h(x) - m_I \} \\ &= \max \left\{ 0, \max_{i=1, \dots, N} (\ell_i(x) - m_I) \right\}. \end{aligned}$$

Since $\mathbb{Q}$ is dense in $\mathbb{R}$ and I is finite, there exist $q^i \in \mathbb{Q}^d$ and $r^i \in \mathbb{Q}$, $i = 1, \dots, N$. Define

$$\tilde{h}(x) := \max \left\{ 0, \max_{i=1, \dots, N} ((q^i)^\top x + r^i) \right\}.$$

The coefficients can be chosen so that

$$\max_{x \in I} \left| g(x) - (\tilde{h}(x) + m_I) \right| < \epsilon.$$

Let D be a positive common denominator of all entries of q^i and r^i . Applying the first part of the proof with $p = N + 1$, $\alpha_i := Dq^i$, and $\beta_i := Dr^i$, and setting $\alpha := 1/D$ and $\beta := m_I$ gives

$$|g(x) - (\alpha f(x) + \beta)| < \epsilon$$

for every $x \in I$.

A.2 Proof of Lemma 2

Consider an iteration in which the algorithm finds an improving step $\tilde{x} \neq 0$. Let $\zeta, \zeta' \in \mathbb{Z}_{\geq 0}^E$ be optimal flows defining $f_z(x)$ and $f_z(x + \tilde{x})$, respectively. Because the improvement is strict and $D_z \cdot (c_z)_e$ is an integer for every $e \in E$,

$$c_z^\top (\zeta - \zeta') \geq \frac{1}{D_z}.$$

For any feasible flow ζ ,

$$|c_z^\top \zeta| \leq m \|u_z\|_\infty \|c_z\|_\infty.$$

The range of feasible objective values is therefore at most $2m \|u_z\|_\infty \|c_z\|_\infty$, so the algorithm performs at most $2m \|u_z\|_\infty \|c_z\|_\infty D_z$ improving iterations.

Each iteration enumerates $O(p^{\lceil \Delta(A_z) \rceil})$ vectors $y \in \mathbb{Z}^p$ satisfying $\|y\|_1 \leq \Delta(A_z)$. Whether $A_z \hat{x} = y$ has an integer solution can be determined in time polynomial in $p, d, \log(1 + \|A_z\|_\infty)$, and $\log(1 + \|y\|_\infty) \leq \log(1 + \lceil \Delta(A_z) \rceil)$ (see, e.g., Schrijver, 1986). Evaluating whether $f_z(x + \hat{x}) < f_z(x)$ requires solving minimum-cost flow problems and can also be done in polynomial time (see, e.g., Tardos, 1985).

It remains to show that the point returned by the algorithm is globally optimal. Suppose that the algorithm terminates at x and that x is not optimal. Among all global minimizers, let x^* minimize $\|A_z x^* - A_z x\|_1$. We must have $A_z x^* \neq A_z x$, since otherwise $f_z(x^*) = f_z(x)$. By the generalized exchange property (G-EXC), there exists $\tilde{x} \in \mathbb{Z}^d$ such that

$$0 < \|A_z \tilde{x}\|_1 \leq \Delta(A_z),$$

$$(A_z \tilde{x})^+ \leq (A_z x^* - A_z x)^+,$$

$$(A_z \tilde{x})^- \leq (A_z x^* - A_z x)^-.$$

and

$$f_z(x^*) + f_z(x) \geq f_z(x^* - \tilde{x}) + f_z(x + \tilde{x}).$$

Because the algorithm found no improvement, $f_z(x) \leq f_z(x + \tilde{x})$. Moreover, $f_z(x^*) \leq f_z(x^* - \tilde{x})$ because x^* is globally optimal. The preceding exchange inequality therefore holds with equality, and $x^* - \tilde{x}$ is also globally optimal. However,

$$\begin{aligned} \|A_z(x^* - \tilde{x}) - A_z x\|_1 &= \|(A_z x^* - A_z x) - A_z \tilde{x}\|_1 \\ &= \|A_z x^* - A_z x\|_1 - \|A_z \tilde{x}\|_1 \\ &< \|A_z x^* - A_z x\|_1, \end{aligned}$$

where the second equality follows from the componentwise bounds above. This contradicts the

choice of x^* and proves that the algorithm returns a global minimizer.

A.3 Proof of Lemma 3

A feasible s - t flow in a directed multi-graph $\hat{G} = (\hat{V}, \hat{E})$ with source node s , sink node t , and arc capacities $\hat{u}$ is a vector $\hat{\Delta} \in [0, \hat{u}]$ satisfying, for every $v \in \hat{V} \setminus \{s, t\}$,

$$\sum_{e \in \delta_v^+} \hat{\Delta}_e - \sum_{e \in \delta_v^-} \hat{\Delta}_e = 0,$$

and

$$\begin{aligned} \sum_{e \in \delta_s^+} \hat{\Delta}_e - \sum_{e \in \delta_s^-} \hat{\Delta}_e \\ = - \sum_{e \in \delta_t^+} \hat{\Delta}_e + \sum_{e \in \delta_t^-} \hat{\Delta}_e. \end{aligned}$$

A well-known result (the flow decomposition theorem) states that any integer feasible s - t flow can be decomposed into unit flows on directed s - t paths and unit circulations on directed cycles; see, e.g., Ahuja et al. (1993).

Consider $y, y' \in \text{dom } f_z^*$ with $y \neq y'$, and let $\xi, \xi' \in \mathbb{Z}_+^m$ be optimal solutions of the minimum-cost flow integer programs defining $f_z^*(y)$ and $f_z^*(y')$, respectively. Let $d := y - y'$ and $\Delta := \xi - \xi' \in \mathbb{Z}^m$.

Auxiliary s - t Network. Create a directed multi-graph $\hat{G}$ on node set $V \cup \{s, t\}$ as follows. First, for each original arc $e = (u, v) \in E$, add $|\Delta_e|$ parallel unit-capacity copies, oriented as:

- (u, v) if $\Delta_e > 0$,
- (v, u) if $\Delta_e < 0$.

Second, add $(d_i)^+$ parallel unit-capacity arcs (s, v_i) and $(d_i)^-$ parallel unit-capacity arcs (v_i, t) for every $i \in [p]$.

Define $\hat{\Delta}$ as the integer s - t flow on $\hat{G}$ that sends one unit of flow on every arc of $\hat{G}$. By construction, for each $i \in [p]$, the contribution of the arc-copies coming from Δ gives net outflow d_i at v_i (whereas all other $v \in V \setminus \{v_1, \dots, v_p\}$ have net outflow 0), and the added arcs (s, v_i) and (v_i, t) contribute net outflow $-(d_i)^+ + (d_i)^- = -d_i$ at v_i . Hence the total net outflow at every $v \in V$ is 0, while

$$\begin{aligned} \sum_{e \in \delta_s^+} \hat{\Delta}_e - \sum_{e \in \delta_s^-} \hat{\Delta}_e &= \sum_{i=1}^p (d_i)^+ =: K, \\ \sum_{e \in \delta_t^+} \hat{\Delta}_e - \sum_{e \in \delta_t^-} \hat{\Delta}_e &= - \sum_{i=1}^p (d_i)^- = -K. \end{aligned}$$

Therefore $\hat{\Delta}$ is a feasible integer s - t flow of value K in $\hat{G}$.

Flow Decomposition and Pairing. By the flow decomposition theorem (Ahuja et al., 1993), we can write

$$\hat{\Delta} = \sum_{k \in P} \Delta^k + \sum_{k \in C} \Delta^k,$$

where each Δ^k is a unit s - t flow supported on a directed s - t path if $k \in P$, and a unit circulation supported on a directed cycle if $k \in C$. Since s has only outgoing arcs and t has only incoming arcs in $\hat{G}$, no directed cycle can include s or t ; hence the entire s - t flow value K is carried by the path flows, i.e., $|P| = K$.

Index the K path flows as $P = \{1, \dots, K\}$. Each unit s - t path Δ^k must start with some arc (s, v_{i_k}) and end with some arc (v_{j_k}, t) . Because there are exactly $(d_i)^+$ parallel arcs leaving s toward v_i (each carrying one unit), the multiset $\{i_1, \dots, i_K\}$ contains each i exactly $(d_i)^+$ times; similarly, $\{j_1, \dots, j_K\}$ contains each j exactly $(d_j)^-$ times. Thus,

$$\begin{aligned} d^+ &= (y - y')^+ = \sum_{k=1}^K e_{i_k}, \\ d^- &= (y - y')^- = \sum_{k=1}^K e_{j_k}. \end{aligned}$$

Define $\tilde{y}_k := e_{i_k} - e_{j_k}$ and, for any $T \subseteq [K]$, $\tilde{y}_T := \sum_{k \in T} \tilde{y}_k$.

Constructing Feasible Flows. Fix $T \subseteq [K]$. For each $k \in T$, consider the internal portion of the s - t path Δ^k between v_{i_k} and v_{j_k} . Define an adjustment vector $\eta^k \in \{-1, 0, 1\}^E$ by setting, for each original arc $e = (u, v) \in E$,

$$\eta_e^k := \begin{cases} -1, & \text{if the path uses a copy oriented as } (u, v), \\ & \text{equivalently, } \Delta_e > 0, \\ +1, & \text{if the path uses a copy oriented as } (v, u), \\ & \text{equivalently, } \Delta_e < 0, \\ 0, & \text{otherwise.} \end{cases}$$

Now define

$$\xi^T := \xi + \sum_{k \in T} \eta^k, \quad \text{and} \quad (\xi')^T := \xi' - \sum_{k \in T} \eta^k.$$

Because the path decomposition uses each arc-copy at most once and there are $|\Delta_e|$ copies attached to each e , the update above modifies ξ_e and ξ'_e by at most $|\Delta_e| = |\xi_e - \xi'_e|$ units on each arc e , and in the direction from the larger to the smaller, so $0 \leq \xi_e^T, (\xi')_e^T \leq (u_z)_e$ for all $e \in E$. Moreover, along the internal nodes of each path the $+1/-1$ changes cancel in the flow-balance constraints, and only the endpoints v_{i_k}, v_{j_k} experience a net change: ξ^T decreases the net supply at i_k by 1 and increases the net supply at j_k by 1. Hence ξ^T is feasible for the problem defining $f_z^*(y - \tilde{y}_T)$, and $(\xi')^T$ is feasible for the problem defining $f_z^*(y' + \tilde{y}_T)$.

Cost Comparison and Conclusion. Finally, by construction the arc-wise changes cancel in the sum of costs:

$$c_z^\top \xi^T + c_z^\top (\xi')^T = c_z^\top \xi + c_z^\top \xi'.$$

Since ξ^T and $(\xi')^T$ are feasible for $y - \tilde{y}_T$ and $y' + \tilde{y}_T$, respectively, we have

$$\begin{aligned} & f_z^*(y - \tilde{y}_T) \\ & \quad + f_z^*(y' + \tilde{y}_T) \\ & \leq c_z^\top \xi^T + c_z^\top (\xi')^T \\ & = c_z^\top \xi + c_z^\top \xi' \\ & = f_z^*(y) + f_z^*(y'), \end{aligned}$$

which proves the desired inequality.

A.4 Proof of Lemma 5

Reminder of Basic Lattice Theory Notions and Results.

Definition A.1 (Lattice). A lattice $\mathcal{L} \subset \mathbb{Z}^p$ is a subgroup of $\mathbb{Z}^p$. That is, for every $x, y \in \mathcal{L}$, $x - y \in \mathcal{L}$.

Definition A.2 ($\mathbb{Z}$ -basis). Given a lattice $\mathcal{L}$. A $\mathbb{Z}$ -basis of $\mathcal{L}$ is a set of vectors $b_1, \dots, b_k \in \mathcal{L}$ that are linearly independent and such that every $y \in \mathcal{L}$ can be written as $y = \lambda_1 b_1 + \dots + \lambda_k b_k$ for some $\lambda_1, \dots, \lambda_k \in \mathbb{Z}$. The *row-stacked* basis matrix

$$B := \begin{pmatrix} b_1^\top \\ \vdots \\ b_k^\top \end{pmatrix} \in \mathbb{Z}^{k \times p}.$$

is called a $\mathbb{Z}$ -basis matrix of $\mathcal{L}$.

Definition A.3 (Orthogonality Defect). Given a lattice $\mathcal{L}$, and a $\mathbb{Z}$ -basis $b_1, \dots, b_k$. Let $b_1^*, \dots, b_k^*$ denote the Gram-Schmidt orthogonalization of $b_1, \dots, b_k$. The orthogonality defect of $b_1, \dots, b_k$ is given by,

$$\gamma(b_1, \dots, b_k) = \prod_{i=1}^k \frac{\|b_i\|_2}{\|b_i^*\|_2}$$

Lemma A.1 (Gram determinant identity; see, e.g., Galbraith (2012)). *Given any linearly independent vectors $b_1, \dots, b_k$, such that $b_1^*, \dots, b_k^*$ is their Gram-Schmidt orthogonalization we have,*

$$\prod_{i=1}^k \|b_i^*\|_2 = \sqrt{\det(BB^\top)}.$$

In particular, for every lattice $\mathcal{L}$ with $\mathbb{Z}$ -basis $b_1, \dots, b_k$, it holds that

$$\prod_{i=1}^k \|b_i\|_2 = \gamma(b_1, \dots, b_k) \cdot \sqrt{\det(BB^\top)}.$$

Lemma A.2 (KZ-reduced basis; see, e.g., Rothvoss (2023); Lagarias et al. (1990)). *Given a lattice $\mathcal{L}$, there exists a $\mathbb{Z}$ -basis $b_1, \dots, b_k$ of $\mathcal{L}$ of orthogonality defect at most k^k . One such basis is the so-called Korkine–Zolotarev (KZ) reduced basis.*

The Last k Rows of V^{-1} Can Be Chosen to Be Any $\mathbb{Z}$ -Basis of $\ker_{\mathbb{Z}}(A_z^T)$. Define

$$\ker_{\mathbb{Z}}(A_z^T) := \{y \in \mathbb{Z}^p : A_z^T y = 0\}.$$

This is a lattice (a subgroup of $\mathbb{Z}^p$). Let $W := V^{-1}$ and let $w_1^\top, \dots, w_p^\top$ be the rows of W .

Lemma A.3. *The rows $w_{r+1}^\top, \dots, w_p^\top$ form a $\mathbb{Z}$ -basis of $\ker_{\mathbb{Z}}(A_z^T)$. Moreover, if $\tilde{w}_1^\top, \dots, \tilde{w}_k^\top$ is any other $\mathbb{Z}$ -basis of $\ker_{\mathbb{Z}}(A_z^T)$, then there exists another SNF decomposition $A_z = V' D U$ such that the last k rows of $(V')^{-1}$ are exactly $\tilde{w}_1^\top, \dots, \tilde{w}_k^\top$.*

Proof. Step 1: the last k rows are independent and lie in the integer kernel:

For $\ell = 1, \dots, k$, $A_z^T w_{r+\ell} = U^\top D^\top V^\top w_{r+\ell} = U^\top D^\top e_{r+\ell} = 0$, because the last k rows of D are zero. Thus $w_{r+\ell} \in \ker_{\mathbb{Z}}(A_z^T)$. They are linearly independent because W is invertible.

Step 2: the last k rows generate $\ker_{\mathbb{Z}}(A_z^T)$ over $\mathbb{Z}$:

Let $y \in \ker_{\mathbb{Z}}(A_z^T)$. Then $0 = y^\top A_z = y^\top V D U$, hence $(y^\top V) D = 0$. Set $\hat{y}^\top := y^\top V \in \mathbb{Z}^{1 \times p}$. Because D has nonzero diagonal only in positions $1, \dots, r$, the equation $\hat{y}^\top D = 0$ forces $\hat{y}_1 = \dots = \hat{y}_r = 0$. Therefore $\hat{y}^\top$ is an integer combination of $e_{r+1}^\top, \dots, e_p^\top$, and hence

$$y^\top = \hat{y}^\top V^{-1}$$

is an integer combination of the last k rows of $V^{-1} = W$.

From Step 1 and 2, $w_{r+1}^\top, \dots, w_p^\top$ is a $\mathbb{Z}$ -basis of $\ker_{\mathbb{Z}}(A_z^T)$.

Step 3: replacing the last k rows by any other $\mathbb{Z}$ -basis $\tilde{w}_1^\top, \dots, \tilde{w}_k^\top$.

Let $\hat{W} \in \mathbb{Z}^{k \times p}$ and $\tilde{W} \in \mathbb{Z}^{k \times p}$ be the row-stacked matrix with rows $w_{r+1}^\top, \dots, w_p^\top$ and $\tilde{w}_1^\top, \dots, \tilde{w}_k^\top$, respectively. Since both $\hat{W}$ and $\tilde{W}$ are $\mathbb{Z}$ -basis matrices of the same lattice (hence can be written as linear integer combinations of each other), there is a unimodular matrix $P \in \mathbb{Z}^{k \times k}$ such that $\tilde{W} = P \hat{W}$. Define the block-diagonal unimodular matrix $S := \begin{pmatrix} I_r & 0 \\ 0 & P \end{pmatrix} \in \mathbb{Z}^{p \times p}$ and set

$$V' := V S^{-1}.$$

Since the last k rows of D are zero, one has $S^{-1} D = D$, hence

$$A_z = V D U = (V S^{-1}) D U = V' D U,$$

so $A_z = V'DU$ is again an SNF of A_z . Finally $(V')^{-1} = SV^{-1}$, so its last k rows are P times the last k rows of V^{-1} , i.e. $\tilde{W}$. $\square$

A Determinant Bound. The following lemma relates $\hat{W}\hat{W}^\top$ to $\det^*(A_z)$ and $\prod_{i=1}^r s_i$. Recall that $\hat{W}$ is the row-stacked matrix of the last k rows of $W = V^{-1}$.

Lemma A.4.

$$\sqrt{\det(\hat{W}\hat{W}^\top)} \leq \frac{\det^*(A_z)}{\prod_{i=1}^r s_i}.$$

Proof. Submatrix notation. For a matrix $M \in \mathbb{R}^{m \times n}$ and index sets $I \subseteq [m]$, $J \subseteq [n]$, we write $M_{[I,J]}$ for the submatrix of M obtained by restricting to rows in I and columns in J . We also use the shorthand $M_{[I,:]} := M_{[I,[n]]}$ for the row-restriction and $M_{[:,J]} := M_{[[m],J]}$ for the column-restriction. If $I \subseteq [p]$, we denote by $I^c := [p] \setminus I$ its complement in $[p] := \{1, \dots, p\}$.

Let $S := \text{diag}_{r,r}(s_1, \dots, s_r) \in \mathbb{Z}^{r \times r}$, so that

$$D = \begin{pmatrix} S & 0 \\ 0 & 0 \end{pmatrix} \in \mathbb{Z}^{p \times d}.$$

Let $\tilde{U} := U_{[1..r,:]} \in \mathbb{Z}^{r \times d}$ be the matrix of the first r rows of U , and let $\tilde{V} := V_{[:,1..r]} \in \mathbb{Z}^{p \times r}$ be the matrix of the first r columns of V . Then

$$A_z = VDU = \tilde{V}S\tilde{U}.$$

Step 1: $\det(\hat{W}\hat{W}^\top) = \det(\tilde{V}^\top \tilde{V})$.

Let $I := \{r+1, \dots, p\}$. By the Cauchy–Binet formula (Horn and Johnson, 2012),

$$\det(\hat{W}\hat{W}^\top) = \sum_{\substack{J \subseteq [p] \\ |J|=k}} \det((V^{-1})_{[I,J]})^2.$$

By Jacobi’s complementary minor identity (Horn and Johnson, 2012), for every $J \subseteq [p]$ with $|J| = |I| = k$,

$$\det((V^{-1})_{[I,J]}) = \frac{\pm \det(V_{[J^c, I^c]})}{\det(V)}.$$

Since V is unimodular, $\det(V) = \pm 1$, and since $I^c = \{1, \dots, r\}$, we have $V_{[J^c, I^c]} = \tilde{V}_{[J^c, :]}$. Therefore,

$$\det((V^{-1})_{[I,J]})^2 = \det(\tilde{V}_{[J^c, :]})^2.$$

As J ranges over all k -subsets of $[p]$, the complements J^c range over all r -subsets of $[p]$. Hence

$$\det(\hat{W}\hat{W}^\top) = \sum_{\substack{I' \subseteq [p] \\ |I'|=r}} \det(\tilde{V}_{[I', :]})^2.$$

Applying Cauchy–Binet again gives

$$\det(\tilde{V}^\top \tilde{V}) = \sum_{\substack{I' \subseteq [p] \\ |I'|=r}} \det(\tilde{V}_{[I',:]})^2,$$

so indeed $\det(\hat{W}\hat{W}^\top) = \det(\tilde{V}^\top \tilde{V})$.

Step 2: comparing $\det^(A_z)$ to $\det(S)\sqrt{\det(\tilde{V}^\top \tilde{V})}$.*

Let $\lambda_1, \dots, \lambda_r > 0$ be the nonzero eigenvalues of $A_z A_z^\top$. By definition, $\det^*(A_z)^2 = \det^*(A_z A_z^\top) = \prod_{i=1}^r \lambda_i$. For a symmetric $p \times p$ matrix of rank r , $\prod_{i=1}^r \lambda_i$ equals the sum of all $r \times r$ principal minors; hence

$$\begin{aligned} \det^*(A_z)^2 &= \sum_{|I'|=r} \det((A_z A_z^\top)_{[I', I']}) \\ &= \sum_{|I'|=r} \det((A_z)_{[I', :]} (A_z)_{[I', :]}^\top). \end{aligned}$$

By Cauchy–Binet, for each such I' ,

$$\det((A_z)_{[I', :]} (A_z)_{[I', :]}^\top) = \sum_{\substack{J \subseteq [d] \\ |J|=r}} \det((A_z)_{[I', J]})^2,$$

and therefore

$$\det^*(A_z)^2 = \sum_{|I'|=r} \sum_{|J|=r} \det((A_z)_{[I', J]})^2.$$

Using $A_z = \tilde{V} S \tilde{U}$, we have for all $|I'| = |J| = r$,

$$(A_z)_{[I', J]} = \tilde{V}_{[I', :]} S (\tilde{U})_{[:, J]},$$

so determinants factor:

$$\det((A_z)_{[I', J]}) = \det(\tilde{V}_{[I', :]}) \det(S) \det((\tilde{U})_{[:, J]}).$$

Squaring and summing separates:

$$\begin{aligned} \det^*(A_z)^2 &= \det(S)^2 \\ &\quad \times \left(\sum_{|I'|=r} \det(\tilde{V}_{[I', :]})^2 \right) \\ &\quad \times \left(\sum_{|J|=r} \det((\tilde{U})_{[:, J]})^2 \right). \end{aligned}$$

Since $\tilde{U}$ has rank r , there exists some J with $\det((\tilde{U})_{[:, J]}) \neq 0$, and this determinant is an integer,

hence $|\det((\tilde{U})_{[:,J]})| \geq 1$. Thus

$$\sum_{|J|=r} \det((\tilde{U})_{[:,J]})^2 \geq 1,$$

and so

$$\begin{aligned} \det^*(A_z)^2 &\geq \det(S)^2 \sum_{|I'|=r} \det(\tilde{V}_{[I',:]})^2 \\ &= \det(S)^2 \det(\tilde{V}^\top \tilde{V}). \end{aligned}$$

Using Step 1 and $\det(S) = \prod_{i=1}^r s_i$, we conclude

$$\det^*(A_z)^2 \geq \left(\prod_{i=1}^r s_i \right)^2 \det(\hat{W} \hat{W}^\top),$$

and taking square-roots proves the claim:

$$\sqrt{\det(\hat{W} \hat{W}^\top)} \leq \frac{\det^*(A_z)}{\prod_{i=1}^r s_i}.$$

□

Conclusion. By Lemma A.3 and Lemma A.2, we may choose the SNF decomposition so that the last k rows of V^{-1} form a $\mathbb{Z}$ -basis $w_{r+1}^\top, \dots, w_p^\top$ of $\ker_{\mathbb{Z}}(A_z^T)$ with $\gamma(w_{r+1}, \dots, w_p) \leq k^k$. The definition of M_i gives $M_i \leq \|w_i\|_2$. Indeed, this is immediate when the coordinate range of w_i is at most 1; otherwise, writing this range as $R_i \geq 2$, we have $\|w_i\|_2 \geq R_i/\sqrt{2} \geq R_i/2 + 1/(4k) = M_i$. Using Lemmas A.1 and A.4, we obtain

$$\begin{aligned} &\prod_{i=r+1}^p M_i \\ &\leq \prod_{i=r+1}^p \|w_i\|_2 \\ &= \gamma(w_{r+1}, \dots, w_p) \sqrt{\det(\hat{W} \hat{W}^\top)} \\ &\leq k^k \sqrt{\det(\hat{W} \hat{W}^\top)} \\ &\leq k^k \frac{\det^*(A_z)}{\prod_{i=1}^r s_i}. \end{aligned}$$

implying the desired result.

□

B Additional Experimental Details

B.1 Predictive-Performance Experiments

B.1.1 Complete Task List

Table B.1 lists the 61 public-data prediction tasks considered before eliminating tasks for which the integer input does not improve validation error over the context alone. Here, K is the number of observations, $d = \dim(x)$, and $q = \dim(z)$.

Table B.1: Complete List of the 61 Public-Data Prediction Tasks Considered Before Filtering

Task	Source	K	(d, q)
Bay Wheels station inventory and stockout risk	(CityBikes contributors, 2025)	11,445	(64, 12)
Bicing station inventory and stockout risk	(CityBikes contributors, 2025)	11,704	(64, 12)
Bluebikes station inventory and stockout risk	(CityBikes contributors, 2025)	11,701	(64, 12)
BPI 2019 purchase-order composition	(van Dongen, 2019)	50,019	(64, 30)
BTS carrier-time departure scheduling and delay	(U.S. Bureau of Transportation Statistics, 2023)	9,696	(128, 46)
BTS airport departure scheduling	(U.S. Bureau of Transportation Statistics, 2023)	5,760	(96, 44)
BTS airport scheduling and departure delay	(U.S. Bureau of Transportation Statistics, 2023)	9,696	(96, 46)
BTS airport departure scheduling and disruptions	(U.S. Bureau of Transportation Statistics, 2023)	9,696	(96, 45)
Capital Bikeshare station inventory and stockout risk	(CityBikes contributors, 2025)	11,661	(64, 12)
Chemical manufacturing process yield	(Kuhn and Johnson, 2013)	176	(45, 12)
CMS nursing-home staffing and inspection quality	(Centers for Medicare & Medicaid Services, 2026)	13,616	(16, 95)
Complete Journey basket composition and return time	(Boehmke and Mortimer, 2019)	150,528	(31, 2781)
Complete Journey soft-drink pricing and revenue	(Boehmke and Mortimer, 2019)	10,816	(64, 41)
Criteo display-ad campaign allocation	(Diemert et al., 2017)	3,162	(64, 104)
Divvy station inventory and stockout risk	(CityBikes contributors, 2025)	11,457	(64, 12)
Dominick’s beer demand	(James M. Kilts Center for Marketing, 2026)	20,319	(64, 81)
Dominick’s beer revenue	(James M. Kilts Center for Marketing, 2026)	20,319	(64, 81)
Dominick’s cheese demand	(James M. Kilts Center for Marketing, 2026)	31,462	(64, 93)
Dominick’s cheese revenue	(James M. Kilts Center for Marketing, 2026)	31,462	(64, 93)

Table B.1 continued.

Task	Source	K	(d, q)
Dominick’s cookie demand	(James M. Kilts Center for Marketing, 2026)	31,110	(64, 93)
Dominick’s cookie revenue	(James M. Kilts Center for Marketing, 2026)	31,110	(64, 93)
Dominick’s frozen-dinner revenue	(James M. Kilts Center for Marketing, 2026)	20,499	(64, 93)
Dominick’s soft-drink demand	(James M. Kilts Center for Marketing, 2026)	31,287	(64, 93)
Dominick’s soft-drink revenue	(James M. Kilts Center for Marketing, 2026)	31,287	(64, 93)
Corporación Favorita product families	(Corporación Favorita et al., 2017)	82,245	(33, 164)
Corporación Favorita selected items	(Corporación Favorita et al., 2017)	48,145	(64, 257)
Corporación Favorita selected-item slice, log-lag context	(Corporación Favorita et al., 2017)	5,465	(64, 251)
Corporación Favorita selected-item slice, raw-lag context	(Corporación Favorita et al., 2017)	5,465	(64, 251)
Corporación Favorita selected-item decision slice	(Corporación Favorita et al., 2017)	4,971	(64, 245)
Genpact food demand	(Analytics Vidhya, 2019)	9,047	(60, 118)
Global grain-legume management and yield	(Cernay et al., 2017)	5,750	(75, 89)
iFLEX dynamic electricity pricing	(Hofmann and Siebenbrunner, 2023)	63,717	(24, 77)
KuaiRec daily content allocation	(Gao et al., 2022)	188,322	(31, 77)
LaDe pickup-request allocation and service time	(Wu et al., 2023)	1,774	(64, 50)
M5 retail prices and revenue	(Makridakis et al., 2022)	19,130	(128, 679)
MatchRN nurse staffing	(Gehri et al., 2023)	1,962	(2, 38)
Retail SKU inventory and next-day demand	(Smith, 2026)	13,080	(64, 52)
Retail SKU inventory and next-day revenue	(Smith, 2026)	13,080	(64, 52)
MIND news-slate recommendation	(Microsoft Research, 2020)	156,965	(64, 71)
Restaurant food-delivery time	(Posit Software, PBC, 2026)	10,012	(27, 9)
NBA home-lineup scoring margin	(Yurko, 2024)	1,208	(18, 99)
NYC yellow-taxi fleet positioning and revenue	(New York City Taxi and Limousine Commission, 2024)	5,111	(64, 71)
Olist e-commerce fulfillment and freight cost	(Olist, 2018)	10,608	(66, 32)
Online Retail II basket composition and return time	(Chen, 2012)	24,567	(64, 45)
Open University learning-activity allocation and achievement	(Kuzilek et al., 2015)	19,198	(73, 50)
Planned delivery-route arcs	(Konovalenko et al., 2026)	19,647	(1260, 555)
Planned delivery-route distance	(Konovalenko et al., 2026)	19,647	(1260, 555)
Planned delivery routes	(Konovalenko et al., 2026)	19,647	(36, 555)
Supermarket POS checkout	(Antczak and Weron, 2019)	3,705	(22, 18)
Rossmann promotion allocation	(Rossmann, 2015)	942	(32, 78)

Table B.1 continued.

Task	Source	K	(d, q)
San Francisco emergency-response allocation	(City and County of San Francisco, 2026)	10,990	(16, 45)
SFpark block pricing and parking occupancy	(San Francisco Municipal Transportation Agency, 2014)	1,222	(32, 38)
Technical-gadget demand	(Cohen et al., 2022)	99	(88, 46)
Bank-marketing contact planning	(Moro et al., 2014)	41,188	(17, 45)
Concrete mixture design	(Yeh, 2007)	1,030	(7, 1)
Diabetes medication planning	(Strack et al., 2014)	101,766	(69, 208)
Garment-production resource allocation	(UCI Machine Learning Repository, 2020)	1,197	(5, 28)
Gas-turbine operating controls and NOx emissions	(Kaya et al., 2019)	36,733	(6, 5)
Superconductor composition design	(UCI Machine Learning Repository, 2018)	21,263	(77, 2)
Walmart markdowns	(Walmart, 2014)	1,395	(5, 57)
Zhao et al. supermarket replenishment and revenue	(Zhao et al., 2020)	10,663	(28, 409)

B.1.2 Hyperparameter Search

Let $d = \dim(x)$. Table B.2 reports the candidate grids from which we draw 12 distinct configurations uniformly at random.

Table B.2: Hyperparameter Grids for the Predictive-Performance Experiments

Model	Hyperparameter	Candidate values
MLP	Number of hidden layers	$\{1, 2\}$
	Hidden-layer width	$\{64, 128, 256, 512\}$
	Learning rate	$\{3 \times 10^{-4}, 10^{-3}, 3 \times 10^{-3}\}$
	Weight decay	$\{0, 10^{-4}\}$
DFN	Nodes per side	$\{d, \max(d, 8), \max(d, 16), \min(\max(d, 32), 128), \min(2d, 128)\}$, with duplicates removed; $\{128\}$ when $d > 128$
	Number of cost-network hidden layers	$\{1, 2\}$
	Cost-network hidden-layer width	$\{64, 128, 256\}$
	Base learning rate	$\{3 \times 10^{-4}, 10^{-3}, 3 \times 10^{-3}\}$
	Weight decay for the context networks	$\{0, 10^{-4}\}$
	Integer-parameter learning-rate multiplier	$\{10, 30, 100\}$
	Initial output-scale factor	$\{0.01, 0.03, 0.1, 0.3\}$
	Scaling-network hidden-layer width	64 (fixed)

For MLP, weight decay is applied to all network parameters. For DFN, it is applied to all

parameters of the cost network and the two output-scaling networks, but not to the latent parameters underlying the integer matrix W and vector h . If η denotes the base learning rate and s the integer-parameter learning-rate multiplier, the context-network parameters are updated with learning rate η , while the latent parameters underlying W and h are updated with learning rate $s\eta$.

The initial output-scale factor r sets the magnitude of the initial context-dependent scale $\alpha(z)$. Specifically, the last layer of the network parameterizing $\alpha(z)$ is initialized so that $|\alpha(z)| = r/\max\{1, \tilde{q}\}$ for every context z , where $\tilde{q}$ is the median positive total mass generated on the training inputs by the near-identity initialization. Architectures with more than five million trainable parameters are removed from the candidate set.

B.1.3 Retained Tasks

Table B.3 describes the 31 tasks retained for the predictive-performance comparison. The reported dimensions are $d = \dim(x)$ and $q = \dim(z)$.

Table B.3: The 31 Tasks Retained for the Predictive-Performance Comparison

Task	Integer input x	Context z and outcome y	Sample size, dimensions, and train/validation/test split
Bay Wheels	Available bicycles at 64 stations	Calendar and time trend; empty or full stations after 60 minutes	$K = 11,445$, $(d, q) = (64, 12)$; 7,936/1,757/1,752
Bicing	Available bicycles at 64 stations	Calendar and time trend; empty or full stations after 60 minutes	$K = 11,704$, $(d, q) = (64, 12)$; 8,195/1,757/1,752
Bluebikes	Available bicycles at 64 stations	Calendar and time trend; empty or full stations after 60 minutes	$K = 11,701$, $(d, q) = (64, 12)$; 8,192/1,757/1,752
BPI 2019 purchase orders	Line-item counts in 64 sub-spend groups	Purchase-order attributes and time; days until the latest invoice clears	$K = 50,019$, $(d, q) = (64, 30)$; 42,466/6,134/1,419
BTS airport scheduling	Scheduled departures in 96 quarter-hour slots	Airport, date, required departures, and recent performance; total departure-delay minutes	$K = 9,696$, $(d, q) = (96, 46)$; 6,752/1,472/1,472
Capital Bikeshare	Available bicycles at 64 stations	Calendar and time trend; empty or full stations after 60 minutes	$K = 11,661$, $(d, q) = (64, 12)$; 8,162/1,749/1,750
Chemical manufacturing	Measurements and settings from 45 process stages	Biological-material measurements; final product yield	$K = 176$, $(d, q) = (45, 12)$; 120/29/27
CMS nursing homes	Employee and contractor staffing levels for eight nursing roles	Facility and resident characteristics; weighted health-survey score	$K = 13,616$, $(d, q) = (16, 95)$; 9,489/2,029/2,098

Table B.3 continued.

Task	Integer input x	Context z and outcome y	Sample size, dimensions, and train/validation/test split
Criteo campaign allocation	Displayed-ad counts for 64 campaigns	User, campaign, and time attributes; attributed conversions	$K = 3,162$, $(d, q) = (64, 104)$; 2,142/408/612
Dominick’s cheese demand	Prices in cents for 64 products	Store, week, and lagged demand; weekly units sold	$K = 31,462$, $(d, q) = (64, 93)$; 21,627/4,908/4,927
Dominick’s cheese revenue	Prices in cents for 64 products	Store, week, and lagged revenue; weekly revenue	$K = 31,462$, $(d, q) = (64, 93)$; 21,627/4,908/4,927
Dominick’s frozen-dinner revenue	Prices in cents for 64 products	Store, week, and lagged revenue; weekly revenue	$K = 20,499$, $(d, q) = (64, 93)$; 14,137/3,195/3,167
Favorita promotions, log-lag context	Promotion indicators for 64 products	Store, calendar, and log-transformed lagged sales; units per transaction	$K = 5,465$, $(d, q) = (64, 251)$; 3,300/769/1,396
Favorita promotions, raw-lag context	Promotion indicators for 64 products	Store, calendar, and raw lagged sales; units per transaction	$K = 5,465$, $(d, q) = (64, 251)$; 3,300/769/1,396
Genpact food demand	Prices and promotion indicators for 20 meals	Fulfillment-center, meal, and calendar attributes; weekly revenue	$K = 9,047$, $(d, q) = (60, 118)$; 6,323/1,338/1,386
Grain-legume management	Crop, variety, input, and management choices	Site, soil, and environmental attributes; crop yield	$K = 5,750$, $(d, q) = (75, 89)$; 4,048/803/899
iFLEX electricity pricing	Hourly electricity prices for 24 hours	Household, calendar, and weather attributes; daily consumption	$K = 63,717$, $(d, q) = (24, 77)$; 43,573/12,006/8,138
Retail inventory	Prior-day inventories for 64 products	Store and calendar attributes; next-day units sold	$K = 13,080$, $(d, q) = (64, 52)$; 9,143/1,955/1,982
MIND news slate	Displayed-article counts in 64 subcategories	Slate size, time, and user history; number of clicks	$K = 156,965$, $(d, q) = (64, 71)$; 95,071/31,624/30,270
Food delivery	Counts of 27 menu items	Day, hour, and distance; delivery time in minutes	$K = 10,012$, $(d, q) = (27, 9)$; 7,007/1,533/1,472
NYC taxi positioning	Available taxis in 64 zones	Calendar and recent system state; next-hour revenue	$K = 5,111$, $(d, q) = (64, 71)$; 3,647/720/744
Olist fulfillment	Daily item quantities by product and seller group	Calendar and order attributes; total freight charge	$K = 10,608$, $(d, q) = (66, 32)$; 7,165/1,738/1,705

Table B.3 continued.

Task	Integer input x	Context z and outcome y	Sample size, dimensions, and train/validation/test split
Open University learning allocation	Activity counts across learning resources and time windows	Student and course attributes; subsequent assessment score	$K = 19,198$, $(d, q) = (73, 50)$; 13,461/2,903/2,834
Planned delivery routes	Planned stop ranks, padded to 36 coordinates	Driver, route, and stop attributes; realized route distance	$K = 19,647$, $(d, q) = (36, 555)$; 13,188/3,575/2,884
SF fire dispatch	Initially dispatched units in 16 categories	Battalion, call mix, priority mix, and calendar; first-unit arrival time	$K = 10,990$, $(d, q) = (16, 45)$; 3,652/3,656/3,682
SFpark pricing	Parking rates in cents for 32 blocks	Calendar and recent occupancy; deviation from the target occupancy band	$K = 1,222$, $(d, q) = (32, 38)$; 856/184/182
Technical-gadget retail	Prices and homepage-placement indicators for 44 products	Calendar and lagged product sales; weekly revenue	$K = 99$, $(d, q) = (88, 46)$; 69/15/15
Bank contact planning	Contact-channel, month, and weekday indicators	Client and campaign history; subscription indicator	$K = 41,188$, $(d, q) = (17, 45)$; 28,831/6,178/6,179
Concrete mixture design	Quantities of seven concrete ingredients	Curing age; compressive strength	$K = 1,030$, $(d, q) = (7, 1)$; 729/155/146
Garment production	Work in progress, overtime, incentive, style changes, and worker units	Team, department, date, and production targets; realized productivity	$K = 1,197$, $(d, q) = (5, 28)$; 823/162/212
Superconductor design	Integer-scaled coefficients for 77 elements	Number of elements and total coefficient; critical temperature	$K = 21,263$, $(d, q) = (77, 2)$; 14,990/3,116/3,157

B.1.4 Downstream Feasible Sets

Table B.4 gives the exact feasible set used for each downstream-optimization task. In the table, $x^{\text{obs}}(z)$ is the integer input observed with context z , and ℓ and u are the coordinatewise minimum and maximum of x in the dataset. The set $\mathcal{P}_j(z)$ contains the positive prices observed for product j in the relevant training market. In the Genpact and technical-gadget tasks, the decision is partitioned as $x = (p, a)$, where p is the price vector and a is the vector of binary promotion or placement decisions. The vector $a^{\text{obs}}(z)$ is the promotion or placement vector observed with context z . For the MIND news task, $s(z)$ is the number of articles in the slate. For the planned-routes task, $m(z)$ is the number of active stops and $\mathfrak{S}_m$ is the set of permutations of $\{0, \dots, m-1\}$. For the

superconductor task, $q(z)$ is the number of constituent elements. We multiply each stoichiometric coefficient by 100,000 to make it integer-valued, and $t(z)$ is the sum of these scaled coefficients. Finally, $\mathcal{G}$ contains the four groups of grain–legume decision coordinates that encode the crop, variety, fertilizer type, and land-preparation method.

Table B.4: Dataset-Specific Feasible Sets for Downstream Optimization

Task	Feasible set $\mathcal{X}(z)$
Bay Wheels	$\{x \in \mathbb{Z}_+^{64} : x \leq u, \mathbf{1}^\top x = \mathbf{1}^\top x^{\text{obs}}(z)\}$
Bicing	$\{x \in \mathbb{Z}_+^{64} : x \leq u, \mathbf{1}^\top x = \mathbf{1}^\top x^{\text{obs}}(z)\}$
Bluebikes	$\{x \in \mathbb{Z}_+^{64} : x \leq u, \mathbf{1}^\top x = \mathbf{1}^\top x^{\text{obs}}(z)\}$
BPI 2019 purchase orders	$\{x \in \mathbb{Z}_+^{64} : x \leq u, \mathbf{1}^\top x = \mathbf{1}^\top x^{\text{obs}}(z)\}$
BTS airport scheduling	$\{x \in \mathbb{Z}_+^{96} : x \leq u, \mathbf{1}^\top x = \mathbf{1}^\top x^{\text{obs}}(z)\}$
Capital Bikeshare	$\{x \in \mathbb{Z}_+^{64} : x \leq u, \mathbf{1}^\top x = \mathbf{1}^\top x^{\text{obs}}(z)\}$
Chemical manufacturing	$\{x \in \mathbb{Z}^{45} : \ell \leq x \leq u\}$
CMS nursing homes	$\{x \in \mathbb{Z}_+^{16} : x \leq u, \mathbf{1}^\top x = \mathbf{1}^\top x^{\text{obs}}(z)\}$
Criteo campaign allocation	$\{x \in \mathbb{Z}_+^{64} : x \leq u, \mathbf{1}^\top x = \mathbf{1}^\top x^{\text{obs}}(z)\}$
Dominick’s cheese demand	$\prod_{j=1}^{64} \mathcal{P}_j(z)$
Dominick’s cheese revenue	$\prod_{j=1}^{64} \mathcal{P}_j(z)$
Dominick’s frozen-dinner revenue	$\prod_{j=1}^{64} \mathcal{P}_j(z)$
Favorita promotions, log-lag context	$\{x \in \{0, 1\}^{64} : \mathbf{1}^\top x \leq 16\}$
Favorita promotions, raw-lag context	$\{x \in \{0, 1\}^{64} : \mathbf{1}^\top x \leq 16\}$
Genpact food demand	$\{(p, a) : p \in \prod_{j=1}^{20} \mathcal{P}_j(z), a \in \{0, 1\}^{40}, \mathbf{1}^\top a = \mathbf{1}^\top a^{\text{obs}}(z)\}$
Grain-legume management	$\{x \in \mathbb{Z}^{75} : \ell \leq x \leq u, x_{1:73} \in \{0, 1\}^{73}, \sum_{j \in G} x_j = 1 \text{ for every } G \in \mathcal{G}\}$
iFLEX electricity pricing	$\{x \in \mathbb{Z}^{24} : \ell \leq x \leq u, \mathbf{1}^\top x = \mathbf{1}^\top x^{\text{obs}}(z)\}$
Retail inventory	$\{x \in \mathbb{Z}_+^{64} : x \leq u, \mathbf{1}^\top x = \mathbf{1}^\top x^{\text{obs}}(z)\}$
MIND news slate	$\{x \in \mathbb{Z}_+^{64} : x \leq u, \mathbf{1}^\top x = s(z)\}$
Food delivery	$\{x \in \mathbb{Z}_+^{27} : x \leq u, \mathbf{1}^\top x = \mathbf{1}^\top x^{\text{obs}}(z)\}$
NYC taxi positioning	$\{x \in \mathbb{Z}_+^{64} : x \leq u, \mathbf{1}^\top x = \mathbf{1}^\top x^{\text{obs}}(z)\}$
Olist fulfillment	$\{x \in \mathbb{Z}_+^{66} : x \leq u, \mathbf{1}^\top x = \mathbf{1}^\top x^{\text{obs}}(z)\}$
Open University learning allocation	$\{x \in \mathbb{Z}_+^{73} : x \leq u, \mathbf{1}^\top x = \mathbf{1}^\top x^{\text{obs}}(z)\}$
Planned delivery routes	$\{x \in \mathbb{Z}^{36} : (x_1, \dots, x_{m(z)}) \in \mathfrak{S}_{m(z)}, x_j = 0 \text{ for } j > m(z)\}$
SF fire dispatch	$\{x \in \mathbb{Z}_+^{16} : x \leq u, \mathbf{1}^\top x = \mathbf{1}^\top x^{\text{obs}}(z)\}$
SFpark pricing	$\prod_{j=1}^{32} \mathcal{P}_j(z)$
Technical-gadget retail	$\{(p, a) : p \in \prod_{j=1}^{44} \mathcal{P}_j(z), a \in \{0, 1\}^{44}, \mathbf{1}^\top a = \mathbf{1}^\top a^{\text{obs}}(z)\}$
Bank contact planning	$\{x \in \{0, 1\}^{17} : \mathbf{1}^\top x_{1:2} = 1, \mathbf{1}^\top x_{3:12} = 1, \mathbf{1}^\top x_{13:17} = 1\}$

Table B.4 continued.

Task	Feasible set $\mathcal{X}(z)$
Concrete mixture design	$\{x \in \mathbb{Z}_+^7 : \ell \leq x \leq u, \mathbf{1}^\top x = \mathbf{1}^\top x^{\text{obs}}(z)\}$
Garment production	$\{x \in \mathbb{Z}^5 : \ell \leq x \leq u, x_1 = x_1^{\text{obs}}(z), x_4 = x_4^{\text{obs}}(z), x_3 \leq x_3^{\text{obs}}(z), x_2 + 240x_5 \leq x_2^{\text{obs}}(z) + 240x_5^{\text{obs}}(z)\}$
Superconductor design	$\{x \in \mathbb{Z}_+^{77} : x \leq u, \mathbf{1}^\top x = t(z), \ x\ _0 = q(z)\}$

B.1.5 Downstream MILP Formulations

DFN. Let r be the number of nodes on each side of the bipartite graph, and let ξ_{ij} be the flow from left node i to right node j . For a fixed context z , minimizing the DFN value $f(x, z; \Pi)$ over $\mathcal{X}(z)$ can be written as

$$\begin{aligned}
\min_{x, \xi} \quad & \sum_{i=1}^r \sum_{j=1}^r c_{ij}(z) \xi_{ij} \\
\text{s.t.} \quad & x \in \mathcal{X}(z), \\
& \sum_{j=1}^r \xi_{ij} = (Wx + h)_i, \quad i = 1, \dots, r, \\
& \sum_{i=1}^r \xi_{ij} = (Wx + h)_j, \quad j = 1, \dots, r, \\
& \xi_{ij} \geq 0, \quad i, j = 1, \dots, r.
\end{aligned} \tag{B.1}$$

Recall that the surrogate function is $\hat{f}(x, z) = \alpha(z)f(x, z; \Pi) + \beta(z)$, with $\alpha(z) > 0$ for minimization tasks and $\alpha(z) < 0$ for maximization tasks. Since $\beta(z)$ is constant for a fixed context, optimizing $\hat{f}(x, z)$ is therefore equivalent to solving (B.1). Together with a MILP representation of $\mathcal{X}(z)$, this gives an exact MILP formulation.

MLP. Consider a trained MLP with L ReLU hidden layers. Let μ_x and σ_x be the training means and standard deviations used to standardize the decision variables; when the decision variables are not standardized, set $\mu_x = 0$ and $\sigma_x = 1$. Let $\tilde{z}$ denote the processed fixed context. We write $\tilde{x}_j = (x_j - \mu_{x,j})/\sigma_{x,j}$ and set $v^0 = (\tilde{x}, \tilde{z})$. Let $s^\ell = B^\ell v^{\ell-1} + d^\ell$ be the input to hidden layer ℓ , and write the scalar network output as $w^\top v^L + b_0$. For every neuron i in layer ℓ , let L_i^ℓ and U_i^ℓ be valid lower and upper bounds on s_i^ℓ over $x \in \mathcal{X}(z)$. Let $\mathcal{M}_\ell$ index the neurons with $L_i^\ell < 0 < U_i^\ell$, let $\mathcal{A}_\ell$ index those with $L_i^\ell \geq 0$, and let $\mathcal{Z}_\ell$ index those with $U_i^\ell \leq 0$. For a fixed context z , the complete

MILP is

$$\begin{aligned}
& \min / \max_{x,s,v,\delta} \quad w^\top v^L + b_0 \\
& \text{s.t.} \quad x \in \mathcal{X}(z), \\
& \quad \tilde{x}_j = \frac{x_j - \mu_{x,j}}{\sigma_{x,j}}, \quad j = 1, \dots, d, \\
& \quad v^0 = (\tilde{x}, \tilde{z}), \\
& \quad s^\ell = B^\ell v^{\ell-1} + d^\ell, \quad \ell = 1, \dots, L, \\
& \quad v_i^\ell \geq s_i^\ell, \quad v_i^\ell \geq 0, \quad i \in \mathcal{M}_\ell, \\
& \quad v_i^\ell \leq s_i^\ell - L_i^\ell (1 - \delta_i^\ell), \quad i \in \mathcal{M}_\ell, \\
& \quad v_i^\ell \leq U_i^\ell \delta_i^\ell, \quad i \in \mathcal{M}_\ell, \\
& \quad \delta_i^\ell \in \{0, 1\}, \quad i \in \mathcal{M}_\ell, \\
& \quad v_i^\ell = s_i^\ell, \quad i \in \mathcal{A}_\ell, \\
& \quad v_i^\ell = 0, \quad i \in \mathcal{Z}_\ell, \quad \ell = 1, \dots, L.
\end{aligned} \tag{B.2}$$

We obtain bounds for the first hidden layer by applying interval arithmetic to the coordinatewise bounds on x implied by $\mathcal{X}(z)$ and propagate these bounds through the subsequent layers. If the outcome was standardized, returning the network output to the original scale is a positive affine transformation and therefore does not change the optimizing decision. This is the standard exact big- M formulation of a ReLU network (Tjeng et al., 2019).